Refinement of Gold: A Metallurgical Analogy for Scientific Manuscript Composition

From Ore to Nine-Nines Purity via Mega-Madlib Token Injection

Daniel Ari Friedman

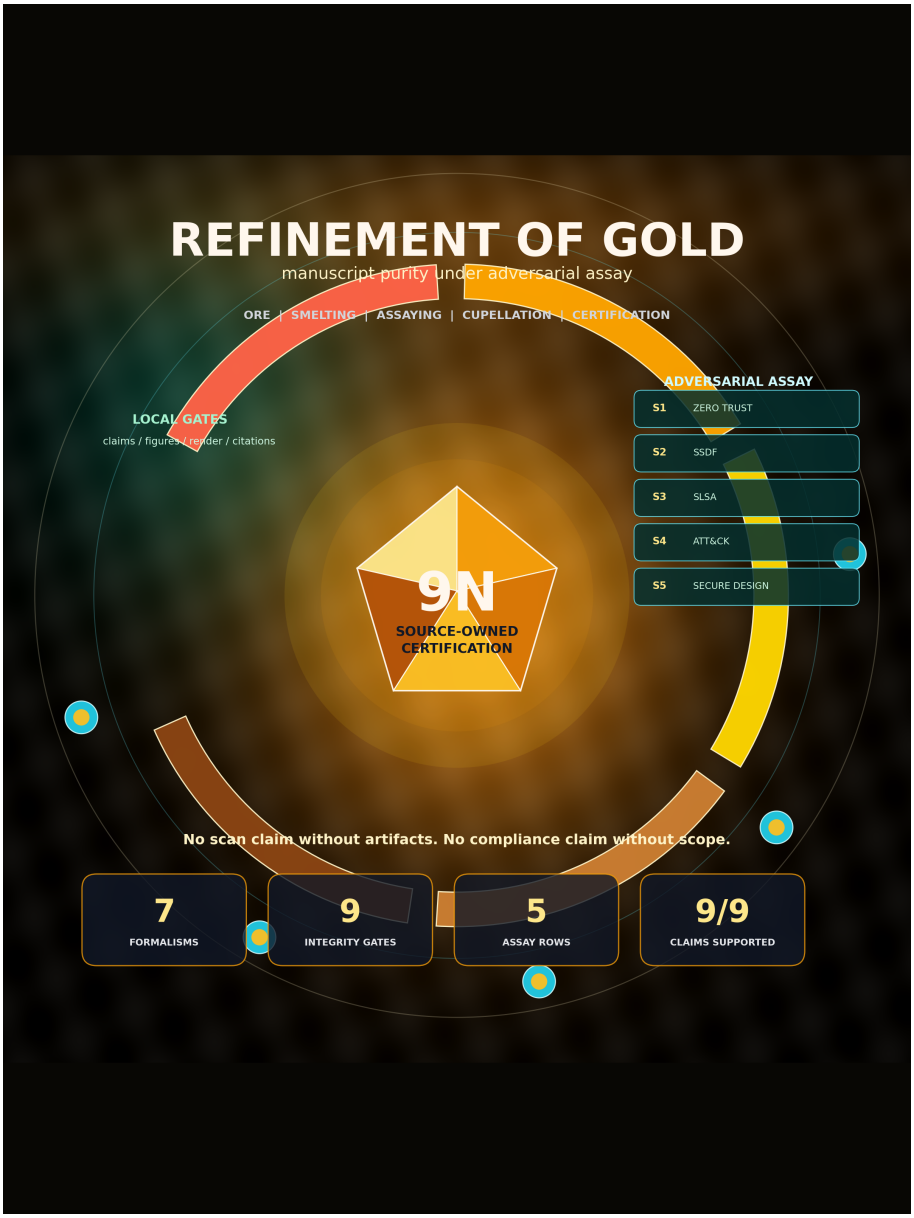
Active Inference Institute

daniel@activeinference.institute

ORCID: 0000-0001-6232-9096

DOI: 10.5281/zenodo.20931955

2026-06-25



Contents

1 Transmission begins **3**

2 Abstract **4**

3 Introduction: Ore to Nine-Nines **5**
3.1 The problem . . . 5
3.2 The analogy as pipeline . . . 5
3.3 Mega-madlib token engine . . . 5
3.4 Implementation circuit . . . 5
3.5 Open question pinned . . . 6

4 Methodology: The Refinery Pipeline **7**
4.1 Research design . . . 7
4.2 Source-domain construction and analogy boundary . . . 7
4.3 Inputs and source ownership . . . 7
4.4 Stage definitions . . . 7
4.5 Refinery model and purity measure . . . 8
4.6 Formalism registry . . . 8
4.7 Deterministic token composition . . . 9
4.8 Config-owned lexicon . . . 9
4.9 Derived grading vocabulary . . . 10
4.10 Execution procedure . . . 10
4.11 Traceability and artifact lineage . . . 11
4.12 Evaluation and acceptance criteria . . . 11
4.12.1 Adversarial assay . . . 11
4.12.2 Scientific-integrity risk model . . . 11
4.12.3 Multi-objective purity . . . 12
4.13 Reproducibility and validity safeguards . . . 12

5 Results: Purity Progression and Karat Grading **13**
5.1 Purity progression . . . 13
5.2 Karat grading scale . . . 14
5.3 Final certification . . . 14
5.4 Token plan summary . . . 14
5.4.1 Category distribution . . . 14
5.4.2 Section distribution . . . 14
5.4.3 Provenance trace . . . 14
5.5 Provenance flow . . . 16
5.6 Purity vs claim support . . . 16
5.7 Token selection sensitivity . . . 16
5.8 Integrity gate matrix . . . 18
5.9 Formalism traceability . . . 18
5.10 Implementation circuit . . . 22
5.11 Claim-evidence assay . . . 22
5.12 Scientific-integrity risk matrix . . . 24
5.13 Evidence-tier ladder . . . 24
5.14 Adversarial security assay . . . 25
5.15 Contribution claims . . . 26
5.16 Shared evidence registry summary . . . 27
5.17 Figure quality report . . . 27

6 Discussion: Load-Bearing vs Rhetorical Analogy **28**
6.1 Load-bearing vs rhetorical . . . 28
6.2 Useful adaptation cases . . . 28
6.3 Misuse modes . . . 28
6.4 Design principles . . . 29
6.5 Analogy-break boundary . . . 30

7 Conclusion: Certification and Forking **31**
7.1 Summary . . . 31
7.2 Forking responsibilities . . . 31

8 Reproducibility: Seeded Regeneration **32**
8.1 Deterministic regeneration . . . 32
8.2 Artifact inventory . . . 32
8.3 Regeneration commands . . . 32
8.4 Config ownership . . . 32
8.5 Evidence-registry separation . . . 33

9 Scope: Related Work and Limitations **34**
9.1 Scope limitations . . . 34
9.2 Related work . . . 34
9.3 Responsible forking . . . 34

10 Quality Probes **36**
10.1 QA probes . . . 36
10.2 Audit rules . . . 36

11 Authoring Contract **38**
11.1 Obligations . . . 38
11.2 Fork checklist . . . 38

12 Transmission ends **40**

1 Transmission begins

This source-owned bookend marks the beginning of the manuscript payload. It is part of the rendered object and carries no claim beyond ordering and payload completeness.

2 Abstract

This paper presents a deterministic, artifact-based methods demonstration that maps gold-refining stages onto a reproducible manuscript pipeline. The executable refinery processes manuscript ore through 5 ordered stages—from raw draft (9K, approximately 37.5% purity) through smelting, assaying, and cupellation—to a local nine-nines certification target (99.9999999%). Pre-1800 accounts ground the relations among extraction, assay, parting, fineness, and marking [Pliny the Elder, c. 77 CE, Biringuccio, 1540, Agricola, 1556, W. B. (William Badcock), 1678, Cramer, 1741]. They do not establish a universal historical sequence or an empirical scale of writing quality.

The analogy is load-bearing in a restricted, testable sense: each stage has an executable owner, ordered input and output states, generated evidence, and a failure condition. A reverse assay identifies the shortest ordered stage prefix that reaches a requested target, while a multi-objective purity vector keeps stage completion, claim support, token provenance, and figure quality distinct. The mega-madlib engine selects 24 configured terms by a seeded SHA-256 digest, recording each choice with its slot, category, ordinal, section, and source path.

The contribution is also formalized: 7 equation-backed formalisms define purity, monotone refinement, token selection, claim support, gate vectors, and certification. This positions the manuscript as a research-compendium artifact in the lineage of literate programming, dynamic reports, and reproducible computational research [Knuth, 1984, Leisch, 2002, Peng, 2011, Sandve et al., 2013]. The project-local claim assay reports 9/9 supported contribution claims (100.00%, passing), while the integrity model exposes 9 source-owned risk dimensions and the shared template evidence registry contributes 960 source-tiered facts when the validation gate has run.

Results: The canonical run reaches 99.9999999% (nine-nines) (24K (nine-nines certified)), with a designed total gain of 90.00%; local nine-nines predicate: Yes. The project-local claim assay reports 9/9 registered claims supported. These are internal workflow results, not estimates of reader-perceived quality, scientific truth, security compliance, or external validity.

Keywords: gold refining, manuscript composition, mega-madlib, token injection, scientific purity, assaying, karat grading, security assay, supply-chain provenance

3 Introduction: Ore to Nine-Nines

Gold refining is one of humanity's oldest purification technologies. The pre-1800 record is not a single modern pipeline, but it gives the analogy a real source domain: Pliny's Book XXXIII treats metals, gold extraction, and touchstone testing as recognizable ancient practices; Biringuccio's *De la Pirotechnia* and Agricola's *De re metallica* turn mining, smelting, cupellation, assaying, and parting into printed technical sequences; Cramer's assay manual codifies the theory/practice split of metal testing [Pliny the Elder, c. 77 CE, Biringuccio, 1540, Agricola, 1556, Cramer, 1741]. Hallmarking regimes make the verification layer socially and legally visible: a seventeenth-century manual for goldsmiths frames "true standard allay," statutes, weights, and counterfeit coin detection as public controls, while the Goldsmiths' Company Assay Office traces the London leopard's-head mark, maker's mark, and date-letter system through pre-1800 milestones [W. B. (William Badcock), 1678, The Goldsmiths' Company Assay Office, 2026]. This paper asks: can that historically plural pipeline serve as a **load-bearing** operational model for scientific manuscript composition - not merely a decorative analogy, but a real mapping from metallurgical stages to template-infrastructure operations?

We operationalize that question through three narrower research questions. **RQ1:** Can each analogical stage be bound to an ordered executable transformation with a source owner and failure condition? **RQ2:** Can prose choices, claims, equations, and figures be regenerated with inspectable provenance? **RQ3:** Can the workflow state its stopping boundary clearly enough that local certification is not mistaken for domain truth, historical authority, or security compliance? The paper answers these questions by constructing and validating one exemplar; it does not compare writing interventions or estimate effects on readers.

3.1 The problem

A scientific manuscript accumulates impurities through its drafting lifecycle: unsupported claims, unresolved references, redundant prose, and citation gaps. The template repository provides infrastructure to detect and remove these impurities - validation gates, cross-reference checks, evidence registries, and coverage enforcement. This aligns with a broader reproducible-research literature that treats code, data, environment, and provenance as first-class scientific materials [Buckheit and Donoho, 1995, Peng, 2011, Stodden et al., 2011, Sandve et al., 2013]. What this exemplar adds is a unifying model that names the purification stages and measures local purity progression.

This exemplar treats source tier and evidence spine as first-class manuscript objects. A claim is not considered refined because it sounds plausible; it is refined when its source path, generated variable, figure label, citation key, and validation gate can all be inspected. That is a narrower claim than empirical truth: reproducibility can set a minimum computational standard without replacing independent replication, domain expertise, or peer review [Peng, 2011, Ioannidis, 2005].

3.2 The analogy as pipeline

We map five gold-refining stages onto manuscript operations:

- 1. ore (9K)
- 2. smelting (18K)
- 3. assaying (22K)
- 4. cupellation (24K)
- 5. certification (24K (nine-nines certified))

Each stage has a metallurgical operation, a manuscript operation, an input purity, and an output purity. Purity increases monotonically, stage order is sequential, and each stage input must equal the preceding output. These invariants are enforced by `src/template_gold_refinement/refinery.py::run_refinery` and `src/template_gold_refinement/purity.py::assert_monotone_increase` and exercised by positive and negative tests. `stages_to_target()` supplies the inverse query: given a target, return the shortest valid prefix rather than selecting later stages out of order.

3.3 Mega-madlib token engine

The manuscript's domain vocabulary is not hand-authored prose but config-owned lexical data, selected deterministically by a seeded SHA-256 digest. The engine generates 24 tokens across 11 slots and 8 lexicon categories. Every token choice is reproducible, traceable to its config key, and bound to a manuscript section. In this respect the exemplar extends literate programming and dynamic statistical reporting: code, data, and prose are kept synchronized, but the prose-level choices are also made inspectable [Knuth, 1984, Leisch, 2002, Marwick et al., 2018].

The deeper token inventory is deliberately spread across the paper. Introduction tokens name the integrity frame; methods tokens bind evidence validation, figure registry check, and citation validation to source-owned operations; results tokens surface artifact manifest, figure registry, and token provenance; discussion tokens mark the fork obligation and domain validator where the analogy must stop.

3.4 Implementation circuit

The metaphor becomes operational only when every transformation has an implementation owner. In this exemplar, configuration creates the ore, `src/template_gold_refinement/refinery.py` defines the purity stages, `src/template_gold_refinement/com`

`position.py` turns slots into deterministic tokens, `src/template_gold_refinement/formalisms.py` owns the equation registry, the `src/figures/` package turns those sources into registered visuals, and the template validators decide whether the hydrated manuscript can be treated as publication metal. The loop is deliberately closed: failures from the validators point back to source files, not to hand-polished output.

3.5 Open question pinned

Is the analogy load-bearing or rhetorical? We assert it is **both**: it frames the methods paper (rhetorical) and operationalizes each stage against real infrastructure (load-bearing). Analogy theory gives the discipline for that claim: preserve relational structure, declare negative analogies, and do not transfer unsupported source-domain properties into the target domain [Gentner, 1983, Hesse, 1966, Holyoak and Thagard, 1995]. The open question is not whether to use the analogy, but where the mapping breaks - a question the discussion addresses.

4 Methodology: The Refinery Pipeline

4.1 Research design

We conducted a deterministic, artifact-based methods demonstration. The research object is one executable manuscript compendium comprising authored configuration and Markdown, project source code, generated data and reports, registered figures, and rendered publication files. The unit of transformation is a refinery stage; the unit of lexical composition is a configured token slot; the unit of evidentiary evaluation is a registered claim or gate. No human participants, stochastic sampling, trained model, or inferential statistical test is involved.

The method asks whether a metallurgical stage structure can be implemented as an inspectable manuscript workflow. It does **not** test whether the resulting purity values predict reader judgments, scientific validity, or editorial acceptance. Accordingly, all reported quantities are properties of this executable exemplar: stage outputs, token selections, registry contents, and validator outcomes.

The implementation has four coupled layers: (1) a 5-stage refinery model, (2) a deterministic mega-madlib token plan, (3) source-owned formalisms and evidence registries, and (4) validation gates that constrain certification language. This separation follows research-compendium practice, in which narrative, code, data, environment, and outputs remain distinct but linked [Marwick et al., 2018]. It also follows executable-paper and notebook traditions in coupling narrative to runnable analysis rather than copying results into prose [Leisch, 2002, Rule et al., 2019]. The narrower contribution here is provenance at the token and claim levels; the workflow does not purport to validate scientific truth.

4.2 Source-domain construction and analogy boundary

Structured reporting guidelines provide the closest scholarly analogue for the gate layer, but they also set its limit. CONSORT, STROBE, PRISMA, ARRIVE, and the EQUATOR Network define field-specific reporting items so a manuscript can be inspected, appraised, and in some cases replicated more easily [Schulz et al., 2010, von Elm et al., 2007, Page et al., 2021, Percie du Sert et al., 2020, EQUATOR Network, 2026]. In this exemplar, token coverage, evidence registration, and render validation play a similar internal role: they make omissions visible and bind declarations to artifacts. They do not test whether an external study was designed well, executed correctly, or substantively true.

The source domain was constructed from pre-1800 descriptions of extraction, smelting or refining, assay, parting/cupellation, fineness, and public marking. Pliny supplies extraction and touchstone context; Biringuccio and Agricola describe staged metallurgical operations; Badcock and the Goldsmiths' Company chronology document standards, weights, statutes, and marks; and Cramer presents assay as a disciplined relation between theory and practice [Pliny the Elder, c. 77 CE, Biringuccio, 1540, Agricola, 1556, W. B. (William Badcock), 1678, The Goldsmiths' Company Assay Office, 2026, Cramer, 1741].

We normalized these historically plural practices into a five-stage relational model. The inclusion criterion was functional relevance to one of five target-domain operations: acquiring draft material, removing evident defects, testing claims, resolving document integrity, or recording a terminal validation state. The ordering preserves the refinement relation needed by the target workflow; it is not evidence that every historical workshop used this sequence or modern chemical theory. The analogy therefore transfers relations among separation, test, refinement, and marking—not regulatory authority, material equivalence, or empirical measures of prose quality [Gentner, 1983, Hesse, 1966].

4.3 Inputs and source ownership

The run begins from three authored inputs: `manuscript/config.yaml`, the Markdown section shells in `manuscript/`, and executable functions in `src/`. Configuration owns the seed, lexicon inventories, slot declarations, contribution claims, audit rules, and security-assay rows. Source modules own stage calculations, token selection, formalism records, evidence aggregation, and figure specifications. Markdown owns interpretation and cross-references but consumes computed values only through generated variables.

Generated files are observations, not editing surfaces. The analysis writes the refinery result, token plan, claim-support registry, integrity summaries, and figure-quality records; manuscript hydration then resolves variables into disposable Markdown. This ownership rule prevents a reported number or selected phrase from being corrected only in the rendered paper while its computational source remains unchanged.

4.4 Stage definitions

#	Stage	Output purity	Karat	Metallurgical operation
1	ore	37.50%	9K	Extract raw gold-bearing ore from the earth
2	smelting	75.00%	18K	Heat ore to separate gold from slag and dross
3	assaying	91.67%	22K	Test a sample to determine gold content and impurities
4	cupellation	99.900%	24K	Refine by blowing air through molten lead-gold alloy
5	certification	99.9999999% (nine-nines)	24K (nine-nines certified)	Certify purity grade and stamp hallmark

4.5 Refinery model and purity measure

The purity sequence across all stages is: 0.100000, 0.375000, 0.750000, 0.916700, 0.999000, 1.000000

Each stage record contains an order, a metallurgical operation, its manuscript analogue, an input purity, and an output purity. The canonical values are design parameters declared in `src/template_gold_refinement/refinery.py`; they are not estimated from a corpus or calibrated against external ratings. Their methodological role is to create a transparent ordinal progression on the bounded interval $[0, 1]$.

The primary invariants are strict monotonicity, sequential stage order, and adjacent-state continuity. `assert_monotone_increase()` raises `ValueError` if a state fails to exceed its predecessor; `run_refinery()` also rejects empty pipelines, nonsequential order, and any stage whose input differs from the preceding output. For stages $s_1, \ldots, s_n$:

$$p_{\text{out}}^{(i)} > p_{\text{in}}^{(i)} \quad \text{and} \quad p_{\text{in}}^{(i+1)} = p_{\text{out}}^{(i)} \quad \forall i \in \{1, \ldots, n-1\}$$

The reverse-assay function `stages_to_target()` returns the shortest ordered prefix whose terminal output reaches a requested purity. Prefix restriction is essential: later stages depend on earlier states and cannot be selected as an unordered set. The full progression is reported in fig. 1 (see sec. 5). Because the values are constructed, the analysis is descriptive and deterministic; no uncertainty interval or significance test is appropriate.

4.6 Formalism registry

The formal layer is generated from `src/template_gold_refinement/formalisms.py`, not hand-numbered prose. tbl. 1 lists the source evidence for each equation, and the equation blocks below are auto-numbered by the renderer.

Table 1: Source-owned formalism registry.

ID	Formalism	Equation	Source
F1	Purity functional	eq. 1	`src/template_gold_refinement/purity.py::format_purity`
F2	Monotone refinement	eq. 2	`src/template_gold_refinement/purity.py::assert_monotone_increase`
F3	Token-selection digest	eq. 3	`src/template_gold_refinement/composition.py::_choose_value`
F4	Claim-support fraction	eq. 4	`src/template_gold_refinement/evidence.py::EvidenceRegistry.support_rate`
F5	Integrity vector	eq. 5	`manuscript/config.yaml#gold_refinement.audit_rules`
F6	Certification predicate	eq. 6	`src/template_gold_refinement/refinery.py::RefineryResult.is_nine_nines_certified`
F7	Adversarial assay	eq. 7	`src/template_gold_refinement/security_assay.py::build_security_assay`

F1: Purity functional. Manuscript purity is treated as a bounded fraction mapped to a reader-facing grade.

$$\pi(s) \in [0, 1], \qquad g(s) = \text{karat}(\pi(s)) \tag{1}$$

The value is descriptive: it summarizes local validation state rather than external quality. Source: `src/template_gold_refinement/purity.py::format_purity`.

F2: Monotone refinement. A valid refinery run requires every stage to improve the previous purity state.

$$\pi_0 < \pi_1 < \cdots < \pi_n \tag{2}$$

The test suite rejects equal or decreasing stage outputs. Source: `src/template_gold_refinement/purity.py::assert_monotone_increase`.

F3: Token-selection digest. Every mega-madlib token is selected from config-owned inventory by a deterministic digest.

$$i = \text{int}(\text{SHA256}(seed \| slot \| category \| ordinal \| inventory)_{0:12}, 16) \bmod |inventory| \tag{3}$$

Changing the seed or inventory changes the plan; replaying both reproduces it. Source: `src/template_gold_refinement/composition.py::_choose_value`.

F4: Claim-support fraction. Contribution claims are assayed by counting supported local evidence pointers.

$$\sigma = \frac{|\{c \in C : supported(c)\}|}{|C|} \tag{4}$$

The numerator and denominator come from the project-local claim-support registry. Source: `src/template_gold_refinement/evidence.py::EvidenceRegistry.support_rate`.

F5: Integrity vector. Scientific integrity is represented as a vector of gate outcomes rather than one scalar badge.

$$\mathbf{v} = (v_{tokens}, v_{figures}, v_{claims}, v_{render}, v_{references}, v_{security}) \tag{5}$$

A publication claim is only as strong as the weakest required gate. Source: `manuscript/config.yaml#gold_refinement.audit_rules`.

F6: Certification predicate. Certification is a predicate over final purity and validation readiness.

$$\text{certified}(r) \iff \pi_{final}(r) \geq 0.999999999 \wedge gates(r) \tag{6}$$

The predicate binds the nine-nines metaphor to the actual validation chain. Source: `src/template_gold_refinement/refinery.py::RefineryResult.is_nine_nines_certified`.

F7: Adversarial assay. Certification requires an explicit adversarial and supply-chain scope, not only ordinary gate success.

$$\text{certified}_{adv}(r) \iff \text{certified}(r) \wedge \forall a \in A_r : \; threat(a) \wedge standard(a) \wedge evidence(a) \wedge validator(a) \wedge boundary(a) \tag{7}$$

The adversarial assay defines scope and evidence requirements; it is not proof of compliance or live scan findings. Source: `src/template_gold_refinement/security_assay.py::build_security_assay`.

4.7 Deterministic token composition

The mega-madlib engine selects tokens from config-owned lexicon categories using a deterministic digest:

$$\text{index} = \text{int}\left(\text{SHA-256}\left(\text{seed} \mid \text{slot} \mid \text{category} \mid \text{ordinal} \mid \text{inventory}\right)[:12], 16\right) \mod n$$

where n is the inventory size. For each declared slot, the procedure concatenates the seed, slot name, category, one-based ordinal, and the complete ordered inventory; hashes the UTF-8 string; converts the first 12 hexadecimal characters to an integer; and reduces it modulo n. Including the complete inventory makes selection sensitive to both membership and order. Replaying the same configuration reproduces the same plan; changing the seed, slot specification, or inventory may change it.

Each selected value is recorded with its variable name, slot, category, section, ordinal, and configuration path. This record permits exact replay and source tracing but does not imply that the selected synonym is semantically superior. Selected metallurgical terms are assaying, parting, and smelting; selected manuscript terms are evidence and evidence. eq. 3 formalizes the rule, while evidence validation, figure registry check, and citation validation name the corresponding validation surfaces.

4.8 Config-owned lexicon

Category	Count	Sample
boundary_terms	5	local claim, analogy boundary, fork obligation…
evidence_terms	5	fact registry, artifact manifest, citation check…
gate_terms	5	prerender, evidence validation, figure registry check…

Category	Count	Sample
integrity_terms	5	evidence spine, source tier, validation gate…
manuscript_terms	5	draft, claim, citation…
metallurgical_terms	5	cupellation, assaying, smelting…
purity_adjectives	5	unrefined, purified, certified…
refinement_verbs	5	assaying, certifying, refining…

4.9 Derived grading vocabulary

Karat grades map purity fractions to a gold-fineness vocabulary used here as an analogy surface. The pre-1800 evidence supports fineness as a regulated testing and marking problem, but not the modern nine-nines target used by this local software predicate [W. B. (William Badcock), 1678, The Goldsmiths' Company Assay Office, 2026, Cramer, 1741, Marsden and House, 2006, London Bullion Market Association, 2026]:

- 9K = 37.5% (ore stage)
- 18K = 75.0% (smelting stage)
- 22K = 91.67% (assaying stage)
- 24K = 99.9% (cupellation stage)
- Nine-nines = 99.9999999% (certification stage)

`src/template_gold_refinement/purity.py::karat_for_purity()` assigns the highest configured grade whose threshold does not exceed the stage purity. The nine-nines label is a deliberately stringent local predicate. It is neither a continuous measure of manuscript quality nor an assertion about a universal gold-market threshold. The grading chart appears in fig. 2 (see sec. 5).

4.10 Execution procedure

Phase	Input	Transformation	Output	Guard
Schema intake	`manuscript/config.yaml`	Load and validate `gold_refinement` block	`GoldRefinementConfig`	config schema tests
Refinery execution	`GoldRefinementConfig`	Run five refinery stages with monotone purity	`RefineryResult`	monotone purity test
Token planning	`GoldRefinementConfig`	Expand slots into deterministic token choices	`TokenPlan`	seed-stability tests
Figure generation	`RefineryResult` and `TokenPlan`	Generate purity progression, karat grading, and token density figures	`../figures/*.png`	nonblank figure tests
Integrity risk modeling	audit rules, failure modes, claims, and shared evidence registry	Score integrity dimensions and summarize evidence tiers	integrity tables and risk visualizations	`tests/test_integrity.py`
Security assay	`gold_refinement.security_assay`	Map adversarial threats and standards to source-owned evidence and claim boundaries	security assay table and variables	`tests/test_security_assay.py`
Manuscript hydration	manuscript shells and `manuscript_variables.json`	Resolve `{{TOKEN}}` placeholders into `output/manuscript/`	hydrated Markdown manuscript	unresolved-token scan
Render and validate	`output/manuscript`	Render PDF, HTML through shared template pipeline	`output/pdf` and `output/web`	render command

For each run, the procedure is:

1. Parse and validate the configuration, including lexicon categories, slots, claims, audit rules, and policy declarations.
2. Execute the canonical stage sequence and reject discontinuous, nonmonotone, empty, or misordered refinery definitions.
3. Generate the token plan and record every selection with its configuration provenance.
4. Build claim-support, integrity, formalism, adversarial-assay, and figure registries from their source owners.
5. Write analysis artifacts and manuscript variables, then hydrate the Markdown section shells.
6. Render publication formats and run reference, evidence, figure, and document validators.
7. Permit certification language only when the required local predicates and gates pass.

The phase table identifies the input, transformation, output, and guard for each step. A fork that modifies a stage or claim must update its source definition, generated variables, visualizations, and validators together.

4.11 Traceability and artifact lineage

The implementation circuit shown in fig. 9 is the method's wiring diagram. It distinguishes three ownership layers. First, authored sources own intent: config declares vocabulary and claims, `src/` owns computation, and the claim ledger registers evidence facts. Second, generated artifacts own observation: token plans, figures, resolved Markdown, reports, and dashboards are rebuilt rather than edited. Third, template gates own permission to promote the manuscript: unresolved tokens, unsupported facts, missing citations, broken references, and invalid PDFs block certification. This is the manuscript analogue of provenance-aware workflow design: entities, activities, agents, and generated outputs are kept traceable so readers can assess reliability rather than infer it from polished prose [Moreau et al., 2013, Belhajjame et al., 2015].

This split keeps the gold metaphor honest. A fork is allowed to change the ore, the furnace, or the assay, but it must do so in the source layer and then let the generated and validation layers expose the consequences.

4.12 Evaluation and acceptance criteria

Evaluation is criterion-based rather than comparative. The run is assessed against predeclared local invariants: complete token hydration, valid stage ordering and monotonicity, registered claim support, resolvable citations and cross-references, registry-aligned figures, successful rendering, and explicit security-claim boundaries. eq. 5 retains these outcomes as a vector so one strong surface cannot compensate for a failed required gate. eq. 4 reports the proportion of locally registered contribution claims with resolvable evidence pointers; it does not score the truth or importance of those claims.

The terminal predicate in eq. 6 requires the configured final purity threshold and successful required gates. Consequently, "certified" means internally consistent and reproducibly generated under this project's declared rules. It does not mean peer reviewed, externally replicated, historically authoritative, secure, or compliant with a reporting standard. Detailed probes and audit rules are reported in sec. 10.

4.12.1 Adversarial assay

The implementation trace handles accidental drift: missing tokens, unsupported claims, malformed citations, stale figures, or broken renders. A security assay adds a different question: could the manuscript sound certified while omitting threat scope, supply-chain provenance, or scan evidence? The assay therefore treats zero trust, secure software development, supply-chain provenance, attack-path modeling, SBOM standards, and secure-by-design guidance as boundary-setting standards rather than proof of compliance [National Institute of Standards and Technology, 2020, 2022, Open Source Security Foundation, 2026, Sigstore, 2026, MITRE, 2026, CycloneDX, 2026, SPDX, 2026, Cybersecurity and Infrastructure Security Agency, 2026].

The assay is implemented as source-owned rows in `gold_refinement.security_assay` and generated records from `src/template_gold_refinement/security_assay.py`. Each row must name a threat, standard or guidance source, local evidence surface, validator, and claim boundary, as specified by eq. 7 and reported in tbl. 5. This study did not run Codex Security or Deep Security Scan and reports no vulnerability findings. Completeness of the assay schema therefore supports scope disclosure, not security compliance — the assay checks only that named threats carry local evidence surfaces and validator rows; it does not certify an absence of vulnerabilities (negative control: removing a row's evidence surface fails schema validation rather than being scored as secure). A row missing any required element is rejected at load time rather than rendered as partial assurance. An assay with zero configured rows reports that no security-scope claim should be made — absence of evidence is rendered as "not configured", never as assurance.

4.12.2 Scientific-integrity risk model

The integrity model converts manuscript risks into source-owned dimensions. It does not replace peer review or domain validation. It names the failure class, severity, detectability, evidence surface, owner, and validator so the manuscript can distinguish "the analogy is vivid" from "the claim is backed by a regenerable check." The current pass reports 9 integrity dimensions; highest residual risk is I4 (Analogy boundary) at 15.

Table 2: Source-owned scientific-integrity dimensions.

ID	Dimension	Residual risk	Owner	Validator
I1	Monotone refinery	4	source code	tests/test_refinery.py
I2	Lexicon completeness	3	config	tests/test_config.py
I3	Token hydration	5	generated variables	tests/test_manuscript_variables.py
I4	Analogy boundary	15	claim ledger	infrastructure.validation.cli evidence –fail-on-issues
I5	Claim support	10	evidence assay	output/reports/claim_support_registry.json
I6	Figure registry	8	figure producer	tests/test_registry_integrity.py
I7	Citation hygiene	4	bibliography	infrastructure.reference.citation validate

ID	Dimension	Residual risk	Owner	Validator
I8	Render readiness	8	template pipeline	template pipeline render and validate stages
I9	Adversarial security assay	15	security assay	tests/test_security_assay.py and manuscript source review

The residual-risk score is an ordinal prioritization heuristic: higher severity and lower detectability raise review priority. It was not fitted to incident data, validated against external judgments, or designed for comparison across projects. Its sole use is to identify where this exemplar—or a fork—needs stronger ownership, evidence, or validation before expanding a claim.

Table 3: Integrity dimensions by owning surface.

Owner	Dimensions
bibliography	1
claim ledger	1
config	1
evidence assay	1
figure producer	1
generated variables	1
security assay	1
source code	1
template pipeline	1

This table also makes generated-number ownership explicit. Counts, support rates, and figure labels belong to regenerated reports and registries; the manuscript consumes them through variables. Authored prose may interpret those values, but it should not silently restate them as hand-maintained facts.

4.12.3 Multi-objective purity

Scalar stage purity is retained only as the state variable of the refinery analogy. `src/template_gold_refinement/purity.py::PurityVector` separately records stage completion, claim support, token provenance, and figure quality on $[0, 1]$. The vector exposes its weakest dimension and a conjunctive all-complete predicate but intentionally defines no weighted average. This prevents a perfect render or terminal stage value from numerically compensating for unsupported claims or missing provenance. Weighting these dimensions would require an external validation study and is outside the present design.

4.13 Reproducibility and validity safeguards

Determinism is assessed by exact replay from the same ordered configuration and source revision. Provenance is assessed by the existence of a path from each reported token, claim, equation, and figure to an owning source and generated record. Negative controls in the test suite exercise malformed configuration and invalid refinery states; figure checks assess file presence, registry parity, dimensions, nonblank content, and color variance. Environment and regeneration details are given in sec. 8.

Four validity limits govern interpretation. First, **construct validity** is limited because purity is a designed workflow state, not a validated measure of writing quality. Second, **external validity** is limited to this exemplar until other domains implement their own mappings and validators. Third, **historical validity** is bounded by the normalized analogy and should not be read as a universal account of metallurgical practice. Fourth, **criterion validity** is local: passing gates demonstrates source ownership and internal consistency, not correctness of domain claims. These limits are part of the method's acceptance rule, not qualifications added after results are known.

5 Results: Purity Progression and Karat Grading

The canonical run completed 5 ordered, continuous stages and reached the configured terminal state of 99.9999999% (nine-nines) (24K (nine-nines certified)). These values verify execution of the declared model; they are not empirical estimates of manuscript quality.

5.1 Purity progression

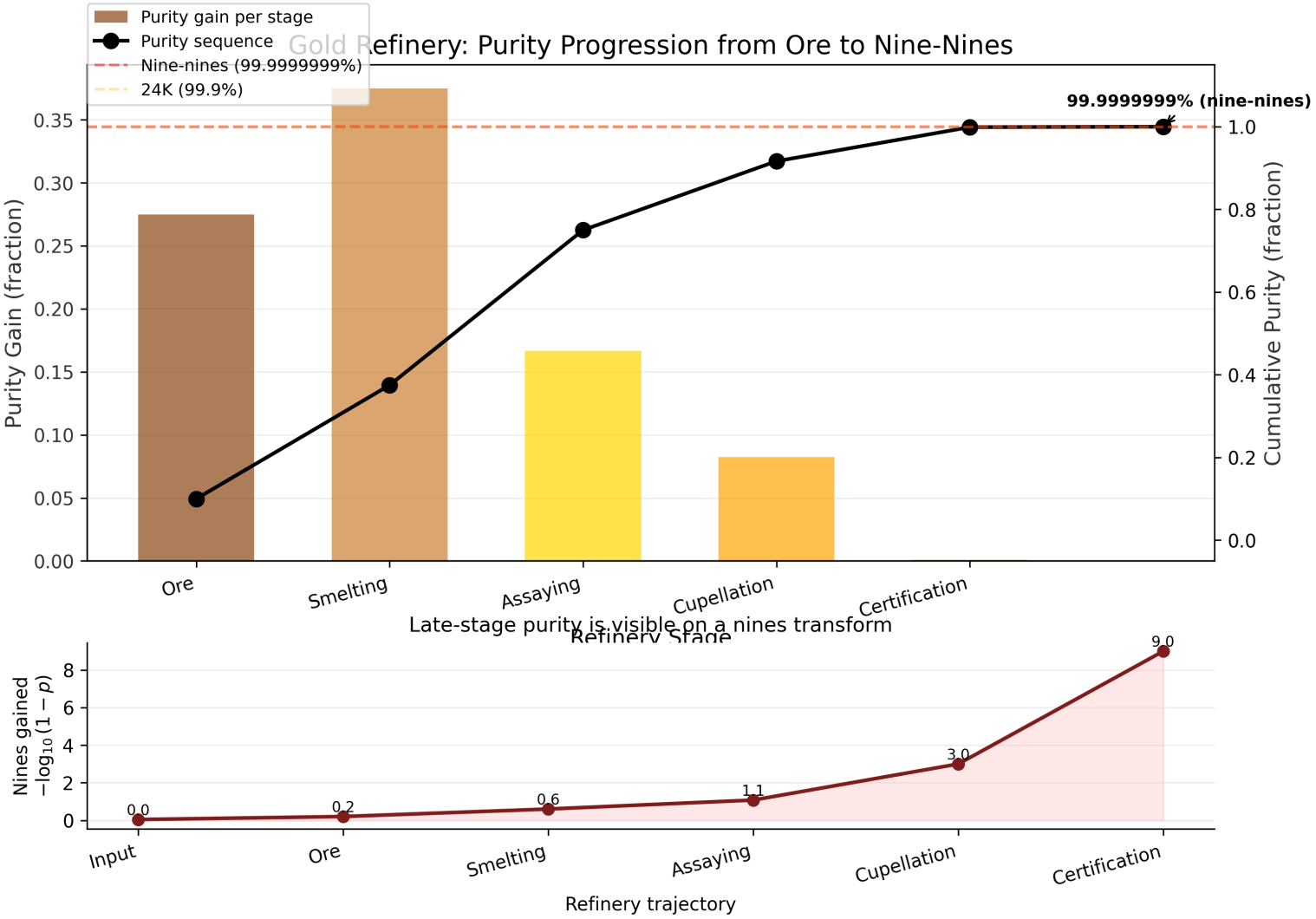


Figure 1: Purity progression across the five refinery stages from ore (9K) to nine-nines certification.

Stage	Name	Output purity	Karat	Gain
1	ore	37.50%	9K	Extract raw gold-bearing ore from the earth
2	smelting	75.00%	18K	Heat ore to separate gold from slag and dross
3	assaying	91.67%	22K	Test a sample to determine gold content and impurities
4	cupellation	99.900%	24K	Refine by blowing air through molten lead-gold alloy

Stage	Name	Output purity	Karat	Gain
5	certification	99.9999999% (nine-nines)	24K (nine-nines certified)	Certify purity grade and stamp hallmark

5.2 Karat grading scale

5.3 Final certification

- **Final purity:** 99.9999999% (nine-nines)
- **Final karat:** 24K (nine-nines certified)
- **Total purity gain:** 90.00%
- **Nine-nines certified:** Yes
- **Nines count:** 9

5.4 Token plan summary

The mega-madlib engine generated 24 tokens from seed 431 across 8 lexicon categories.

5.4.1 Category distribution

Category	Count
boundary_terms	4
evidence_terms	5
gate_terms	5
integrity_terms	2
manuscript_terms	2
metallurgical_terms	3
purity_adjectives	2
refinement_verbs	1

5.4.2 Section distribution

Section	Token count
authoring_contract	2
discussion	3
evaluation	2
introduction	2
methodology	8
reproducibility	2
results	5

5.4.3 Provenance trace

Variable	Category	Value	Section	Source
AUTHORING_BOUNDARY_TERM_1	boundary_terms	analogy boundary	authoring_contract	manuscript/config.yaml#gold_refinement.lexicon.boundary_terms[1]
AUTHORING_BOUNDARY_TERM_2	boundary_terms	non-claim	authoring_contract	manuscript/config.yaml#gold_refinement.lexicon.boundary_terms[4]
DISCUSSION_BOUNDARY_TERM_1	boundary_terms	fork obligation	discussion	manuscript/config.yaml#gold_refinement.lexicon.boundary_terms[2]

Variable	Category	Value	Section	Source
DISCUSSION_BOUNDARY_TERM_2	boundary_terms	domain validator	discussion	manuscript/config.yaml#gold_refinement.lexicon.boundary_terms[3]
DISCUSSION_REFINEMENT_VERB	refinement_verbs	smelting	discussion	manuscript/config.yaml#gold_refinement.lexicon.refinement_verbs[3]
EVALUATION_GATE_TERM_1	gate_terms	prerender	evaluation	manuscript/config.yaml#gold_refinement.lexicon.gate_terms[0]
EVALUATION_GATE_TERM_2	gate_terms	citation validation	evaluation	manuscript/config.yaml#gold_refinement.lexicon.gate_terms[3]
INTRO_INTEGRITY_TERM_1	integrity_terms	source tier	introduction	manuscript/config.yaml#gold_refinement.lexicon.integrity_terms[1]
INTRO_INTEGRITY_TERM_2	integrity_terms	evidence spine	introduction	manuscript/config.yaml#gold_refinement.lexicon.integrity_terms[0]
METHOD_GATE_TERM_1	gate_terms	evidence validation	methodology	manuscript/config.yaml#gold_refinement.lexicon.gate_terms[1]
METHOD_GATE_TERM_2	gate_terms	figure registry check	methodology	manuscript/config.yaml#gold_refinement.lexicon.gate_terms[2]
METHOD_GATE_TERM_3	gate_terms	citation validation	methodology	manuscript/config.yaml#gold_refinement.lexicon.gate_terms[3]
METHOD_MANUSCRIPT_TERM_1	manuscript_terms	evidence	methodology	manuscript/config.yaml#gold_refinement.lexicon.manuscript_terms[4]
METHOD_MANUSCRIPT_TERM_2	manuscript_terms	evidence	methodology	manuscript/config.yaml#gold_refinement.lexicon.manuscript_terms[4]
METHOD_METAL_TERM_1	metallurgical_terms	assaying	methodology	manuscript/config.yaml#gold_refinement.lexicon.metallurgical_terms[1]
METHOD_METAL_TERM_2	metallurgical_terms	parting	methodology	manuscript/config.yaml#gold_refinement.lexicon.metallurgical_terms[3]
METHOD_METAL_TERM_3	metallurgical_terms	smelting	methodology	manuscript/config.yaml#gold_refinement.lexicon.metallurgical_terms[2]
REPRO_EVIDENCE_TERM_1	evidence_terms	fact registry	reproducibility	manuscript/config.yaml#gold_refinement.lexicon.evidence_terms[0]

Variable	Category	Value	Section	Source
REPRO_EVIDENCE_TERM_2	evidence_terms	figure registry	reproducibility	manuscript/config.yaml#gold_refinement.lexicon.evidence_terms[3]
RESULTS_EVIDENCE_TERM_1	evidence_terms	artifact manifest	results	manuscript/config.yaml#gold_refinement.lexicon.evidence_terms[1]
RESULTS_EVIDENCE_TERM_2	evidence_terms	figure registry	results	manuscript/config.yaml#gold_refinement.lexicon.evidence_terms[3]
RESULTS_EVIDENCE_TERM_3	evidence_terms	token provenance	results	manuscript/config.yaml#gold_refinement.lexicon.evidence_terms[4]
RESULTS_PURITY_ADJ_1	purity_adjectives	unrefined	results	manuscript/config.yaml#gold_refinement.lexicon.purity_adjectives[0]
RESULTS_PURITY_ADJ_2	purity_adjectives	purified	results	manuscript/config.yaml#gold_refinement.lexicon.purity_adjectives[1]

Selected purity adjectives for this section: unrefined, purified. Selected evidence terms: artifact manifest, figure registry, token provenance.

5.5 Provenance flow

The provenance flow in fig. 4 makes the refinement analogy auditable as a directed source path rather than a decorative metaphor. The graph starts from the same stage sequence used in the purity table and carries that sequence forward to certification, with edge width proportional to the purity gain owned by `src/template_gold_refinement/refinery.py::run_refinery`. A reader can therefore ask where each improvement enters the pipeline and whether it is supported by the same source that generated the reported purity numbers.

The main result is structural: certification is not allowed to appear as a terminal label detached from the intermediate stages. It is only reached after the upstream transformations have been generated, ordered, and connected. This matters for the manuscript contract because provenance is doing more than recording file paths. It is preserving causal custody from raw material through assay and final certification, so a later change to the stage model must move through the same graph, table, and validation surfaces.

5.6 Purity vs claim support

The purity-versus-claim-support view in fig. 5 places two differently scoped measurements on explicit axes: stage output purity on the horizontal axis and the single project-level contribution-claim assay on the vertical axis. The same observed support rate is therefore repeated across stages. The figure does not fabricate a stagewise claim-support trajectory from a project-level aggregate.

In the current generated assay, 9 of 9 contribution claims are supported (100.00%). The plot is diagnostic rather than correlational: five stage states and one ledger-level rate do not constitute independent observations suitable for association testing. Its purpose is to reveal disagreement between a late refinery state and weak overall claim support without combining the two into one score.

5.7 Token selection sensitivity

The token selection heatmap in fig. 6 turns the mega-madlib engine into an inspectable sensitivity surface. The manuscript uses seed 431 for the reported token plan, but the figure asks a neighboring question: how do selected inventory indices move when seeds and lexicon categories vary? This is not a stochastic robustness claim. It is a deterministic audit of the digest rule in `src/template_gold_refinement/composition.py::generate_token_plan` against the configured lexicon inventories in `manuscript/config.yaml`.

This view separates three issues that prose alone tends to blur. First, token injection is reproducible: the same seed and inventory generate the same choices. Second, the available vocabulary is a real input surface: a thin or unbalanced category would be visible as a constrained selection band. Third, render-time hydration is not silently sampling new language. The heatmap therefore protects the manuscript from a common generative-writing failure mode, where a polished section appears stable but its wording is actually

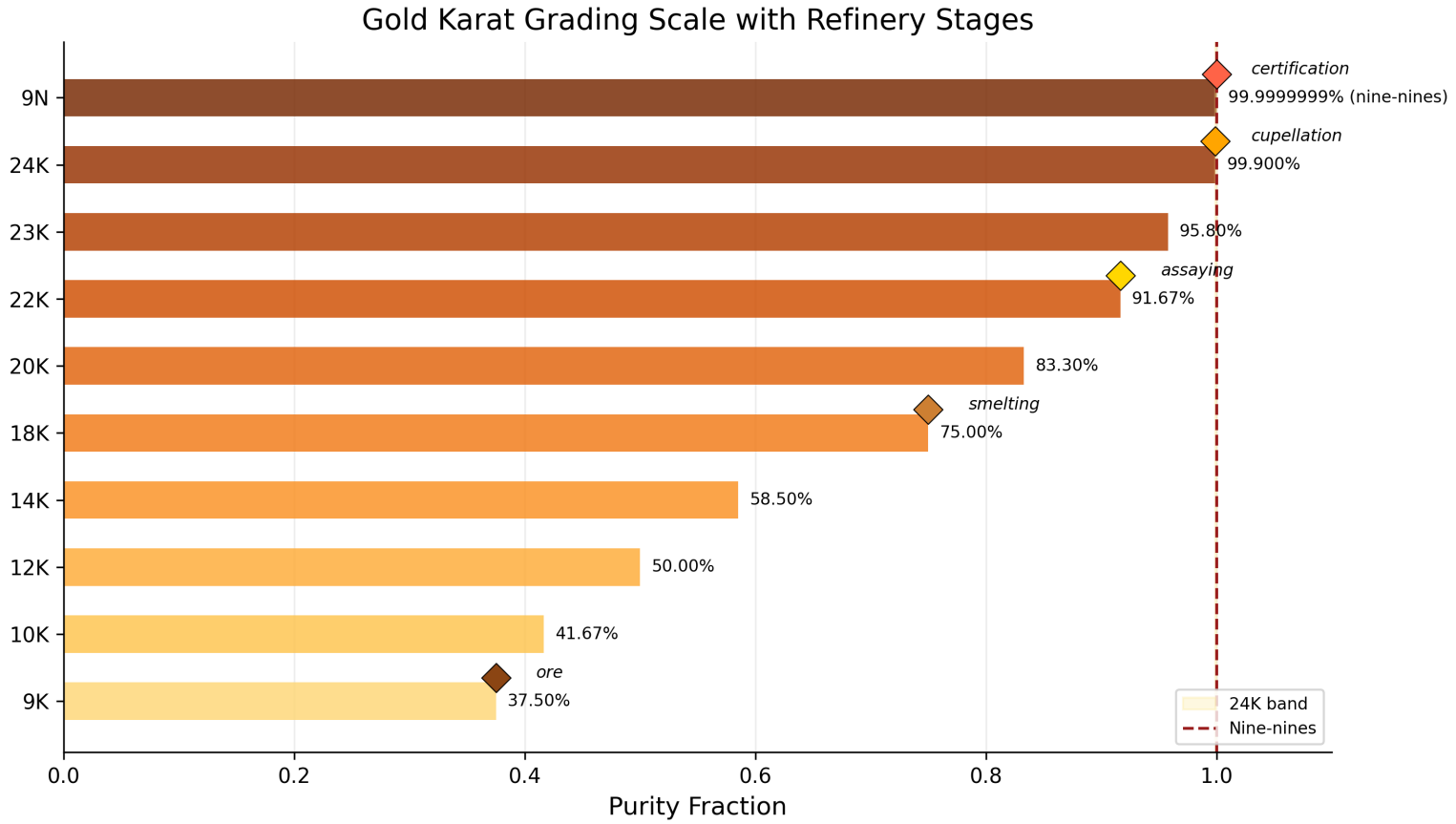


Figure 2: Gold karat grading scale (9K–24K + nine-nines) with refinery stage markers.

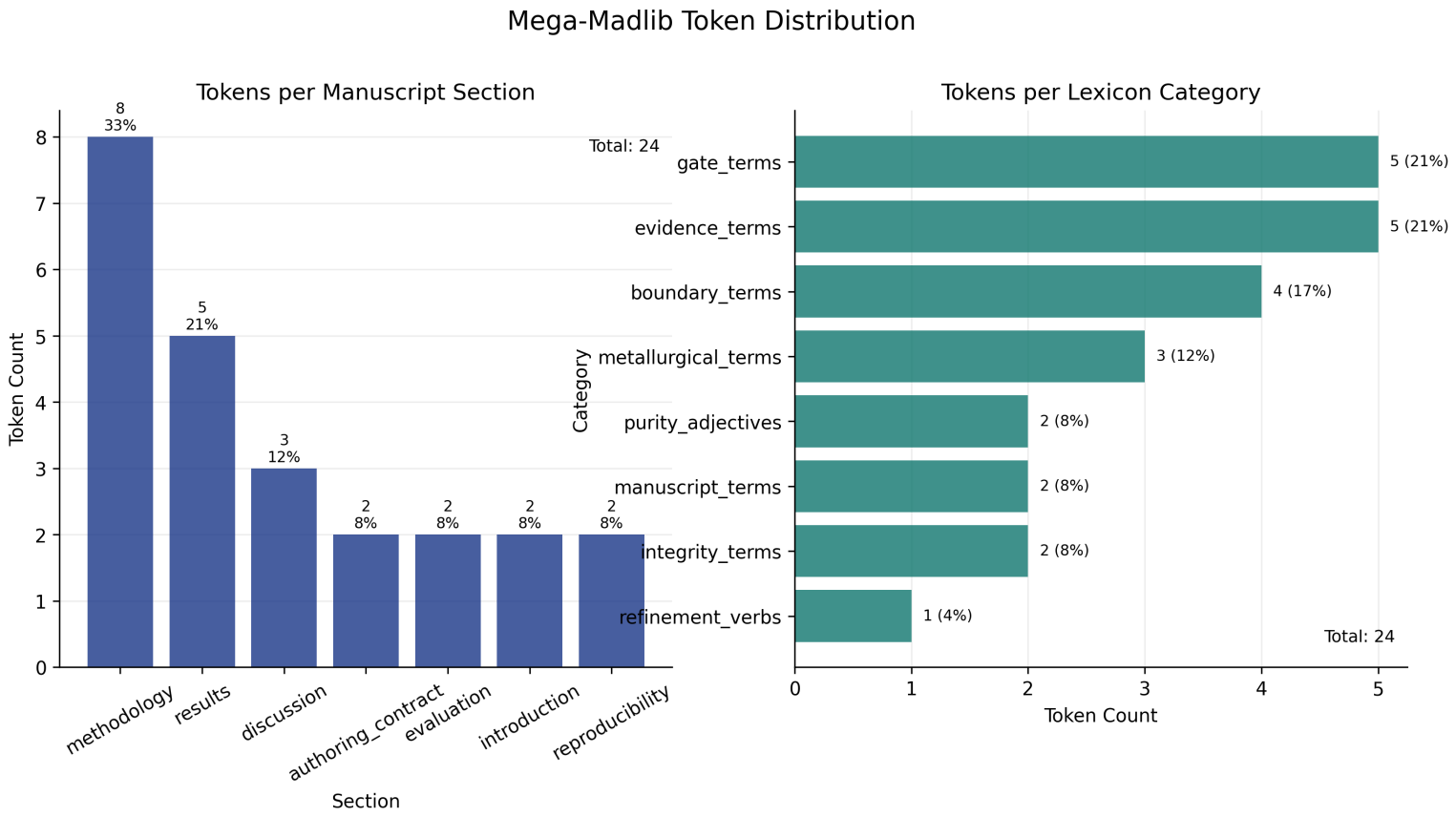


Figure 3: Mega-madlib token distribution across manuscript sections and lexicon categories.

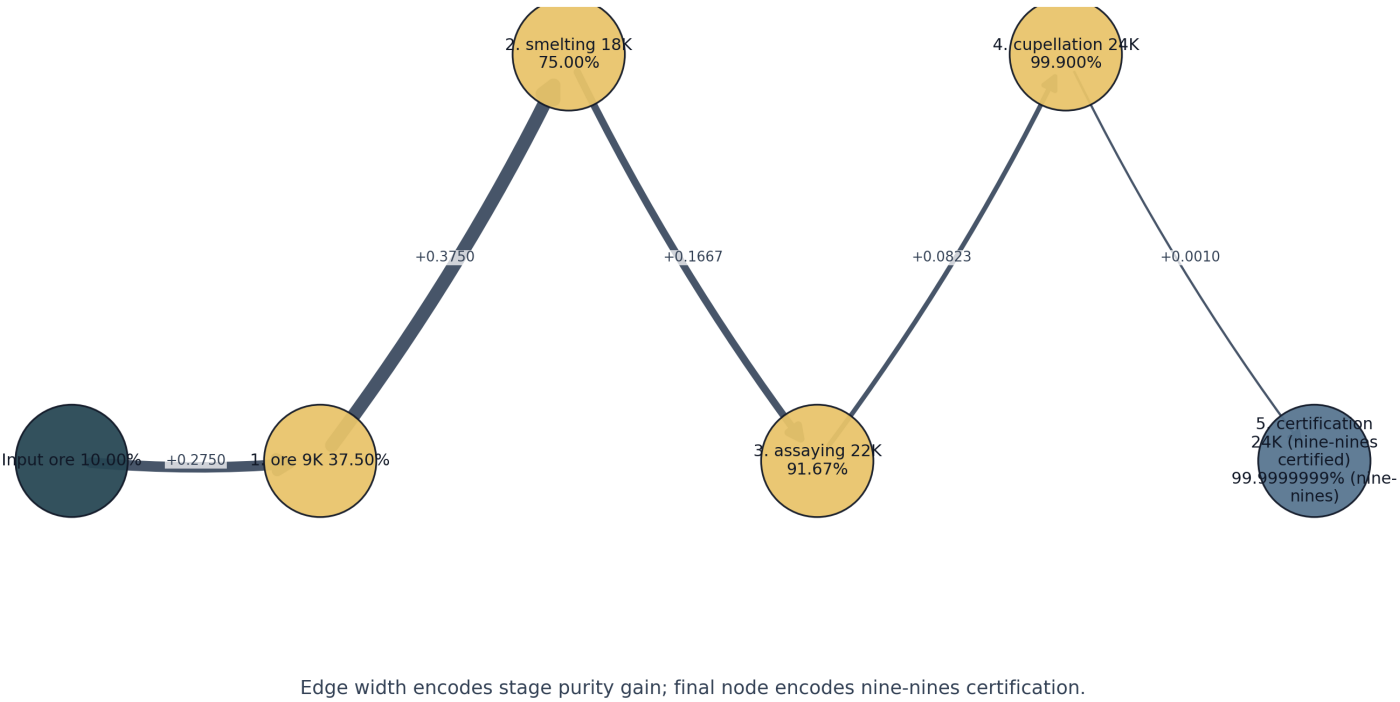


Figure 4: Provenance trace: ore → stages → certification purity flow.

controlled by hidden or late-bound choices. The result is a sensitivity check on authoring machinery, not a claim that any particular synonym is scientifically superior.

5.8 Integrity gate matrix

The integrity gate matrix in fig. 7 makes the validation story visible: audit rules are not prose promises unless they connect to tests, manuscript surfaces, and generated artifacts.

Each row begins as a configured audit rule, but the matrix asks whether that rule has enough contact with the project to matter. A rule that names a risk but does not touch tests, manuscript text, or generated outputs remains advisory. A rule that reaches all three surfaces becomes enforceable: tests can fail, manuscript hydration can expose stale or missing variables, and generated reports can show whether the artifact exists. The matrix is therefore a map of operational coverage, not a checklist of intentions.

This is the validation story that supports the broader purity analogy. Smelting can remove obvious dross, but scientific-integrity failures often survive as well-written unsupported claims, stale tables, or figures that no longer match their source data. By separating missing, partial, and full coverage across gate surfaces, fig. 7 identifies where a future author would need to add a test, generated artifact, or manuscript variable before strengthening a claim. The result is intentionally local to this exemplar: coverage is measured against the specific audit rules configured here, not against a universal publication-readiness standard.

5.9 Formalism traceability

The formalism traceability view in fig. 8 links each equation-backed formalism to the source surface that owns it. This is the visual counterpart to tbl. 1.

The registry currently exposes 7 source-owned formalisms. The figure makes their ownership legible by linking each formalism to its equation identifier and to the source surface that emits it. That linkage is important because equation labels can otherwise create a false sense of rigor: a numbered equation looks formal even when its assumptions, variables, and implementation owner are not recoverable. Here the formal object must remain connected to `src/template_gold_refinement/formalisms.py`, the generated registry table, and the manuscript reference that consumes it.

The graph also helps distinguish formal support from decorative notation. A formalism earns a place in the manuscript only when it names a claim boundary or computation that the source can regenerate. If an equation is added without a source owner, it should fail this traceability pattern before it becomes part of the results narrative. Conversely, if the source registry changes, the visual traceability layer should change with it. That is the desired behavior: the formal layer is a generated contract, not hand-maintained mathematical ornamentation.

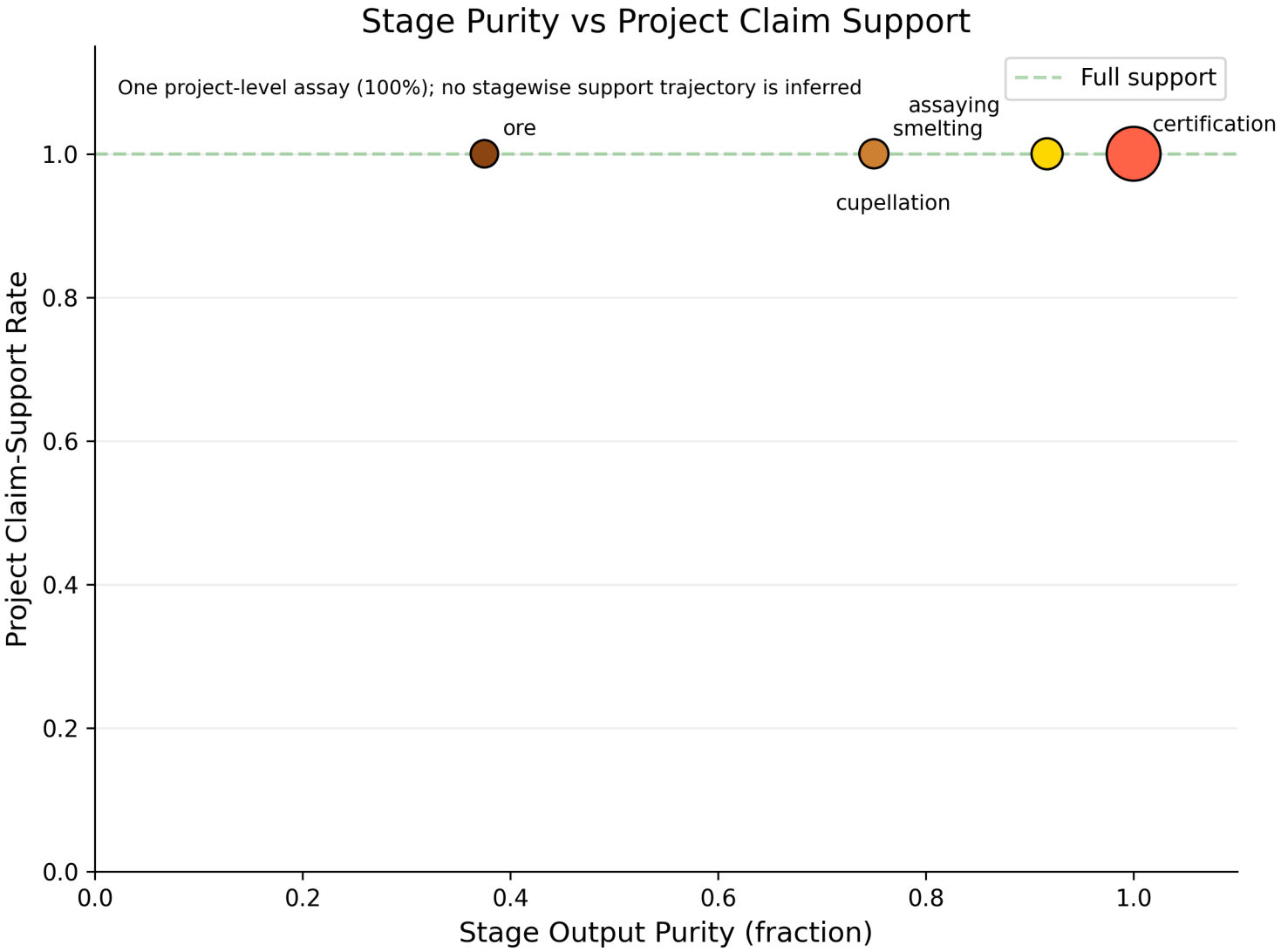


Figure 5: Stage purity plotted against the single project-level claim-support assay.

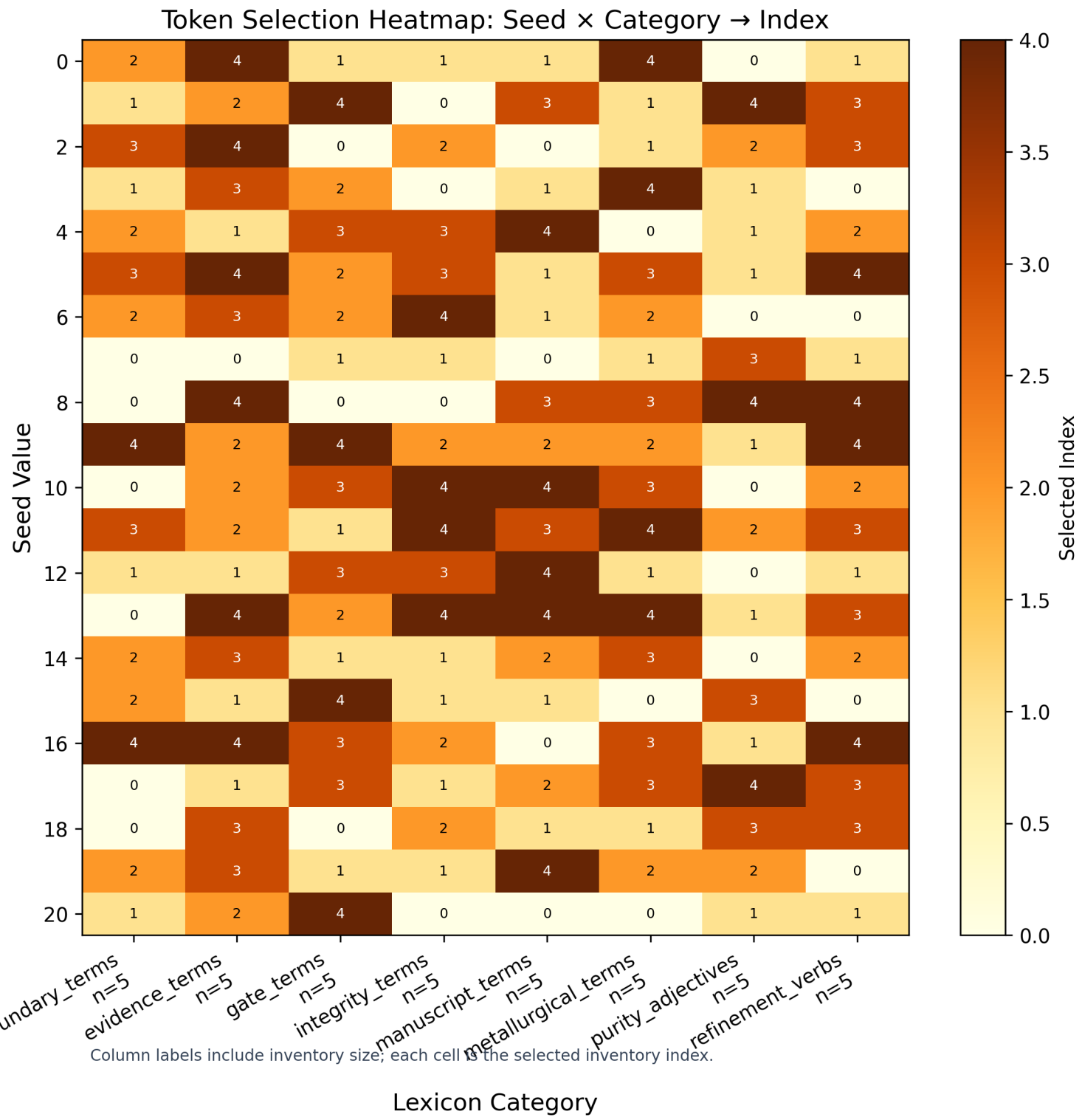


Figure 6: Token selection heatmap: seed × category → selected index.

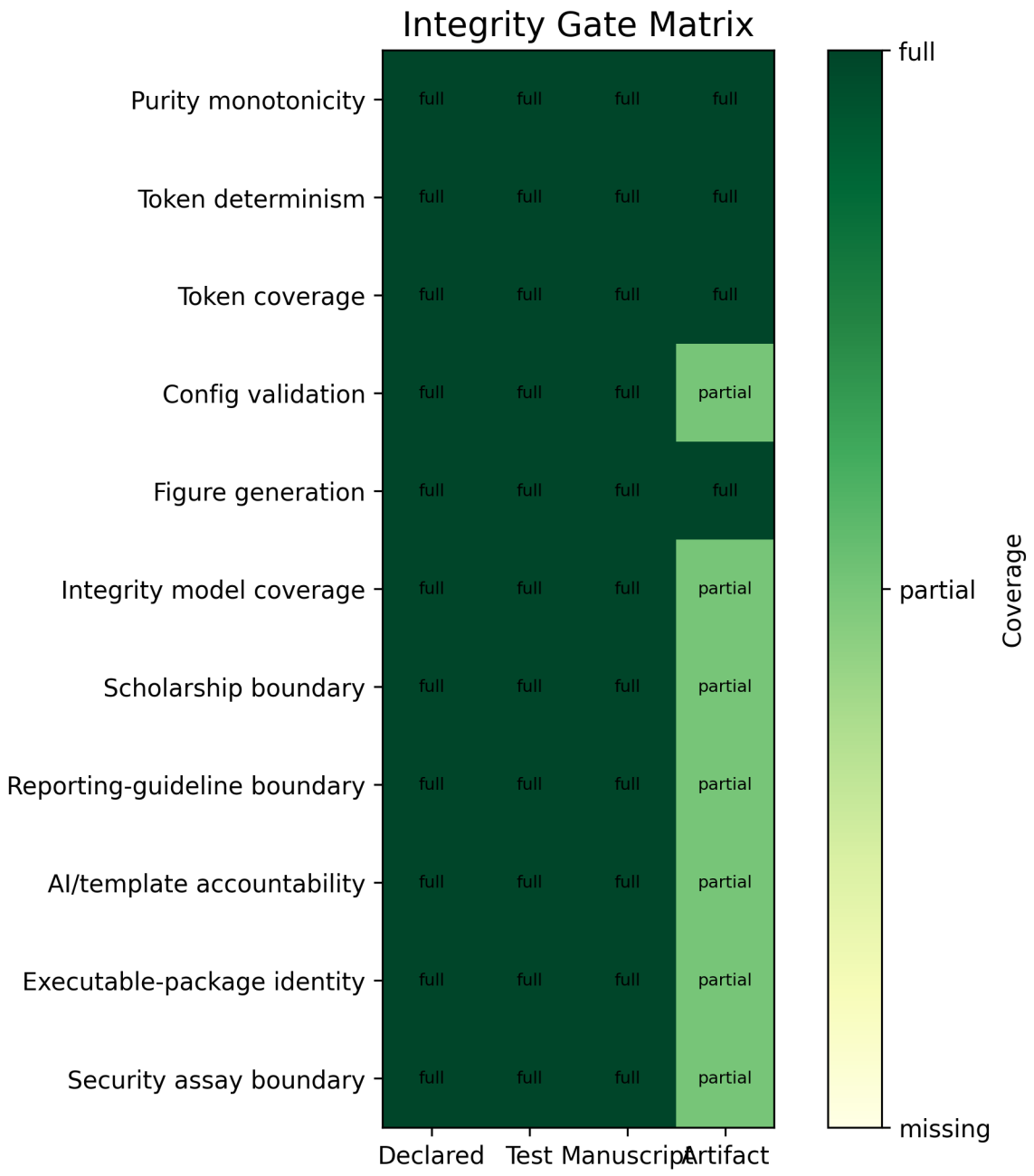


Figure 7: Integrity-gate matrix linking audit rules to tests, manuscript surfaces, and generated artifacts.

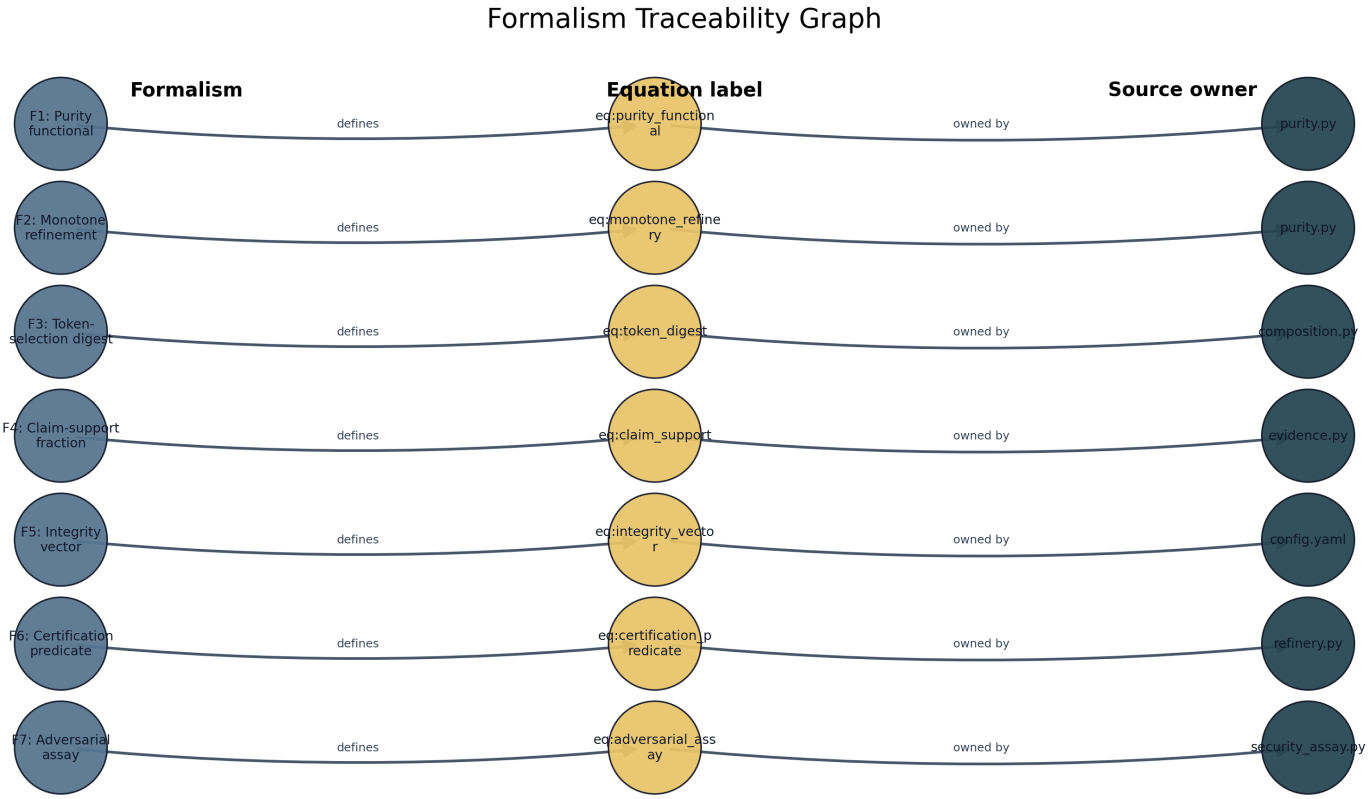


Figure 8: Formalism traceability from source-owned equation identifiers to source evidence.

5.10 Implementation circuit

The implementation circuit in fig. 9 shows how the concept is executed rather than merely described. Configuration feeds code; code emits variables, figures, and reports; manuscript hydration consumes those artifacts; validators feed errors back to source ownership. The figure is intentionally circular because the artifact is not complete after prose generation. It is complete only after the validation return path has no blocking evidence, citation, reference, or render failures.

The circuit is the results section's strongest guard against a prose-only interpretation of the template. It shows four layers that must remain connected: configuration, project code, generated artifacts, and validation feedback. A change in `manuscript/config.yaml` is not complete when the file is saved. It must pass through source functions, generate updated variables and figures, hydrate the manuscript, and survive the validator return path. The circular layout is therefore a process claim: the manuscript is complete only when the loop closes without unresolved failures.

This view also explains why the exemplar treats generated output as disposable but not optional. Output files are not edited by hand, yet they are the evidence surface through which the reader and validators inspect the source contract. When a validator reports a broken citation, missing figure, stale variable, or unsupported claim, the circuit points backward to the owning source surface rather than encouraging local patching of rendered Markdown. That direction of repair is central to the manuscript's definition of refinement.

5.11 Claim-evidence assay

The claim-evidence assay in fig. 10 turns the assaying stage into a reader-facing diagnostic. Each bar is a contribution claim from `manuscript/config.yaml`, and each annotation names the source file or symbol used to support it. This makes the contribution ledger inspectable at the same level as the purity plots: unsupported claims would appear as failed assays rather than remaining hidden in prose.

The generated assay currently reports 9 supported claims out of 9. The value of the figure is not the perfect score by itself; it is the way the score is forced to name its evidence surface and boundary. A contribution claim is not merely present in prose. It must be registered, matched to supporting evidence, and assigned a boundary that tells the reader what the support does not cover. This keeps contribution language from drifting beyond the local evidence available in the project.

The bar-plus-topology design makes two failure modes visible. If a claim lacks support, the bar view would show the failed assay directly. If a claim is supported only within a narrow scope, the graph side still preserves the boundary classification rather than flattening the result into a binary pass. That matters for the security and integrity extensions in this manuscript: a claim can be source-owned without becoming a compliance claim, and a generated assay can confirm local evidence without pretending to certify the wider supply chain.

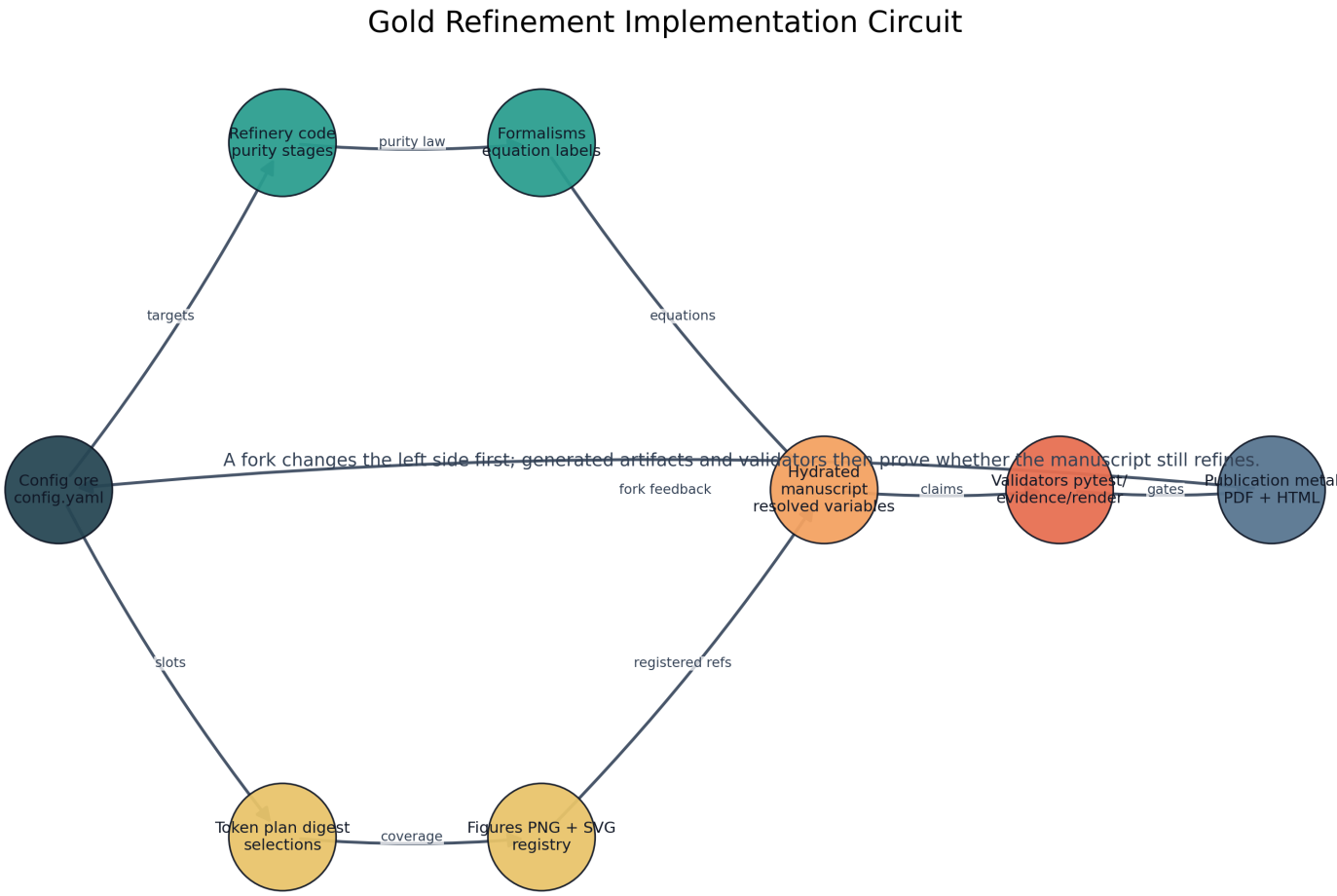


Figure 9: Implementation circuit from config-owned ore through generated manuscript artifacts and validation feedback.

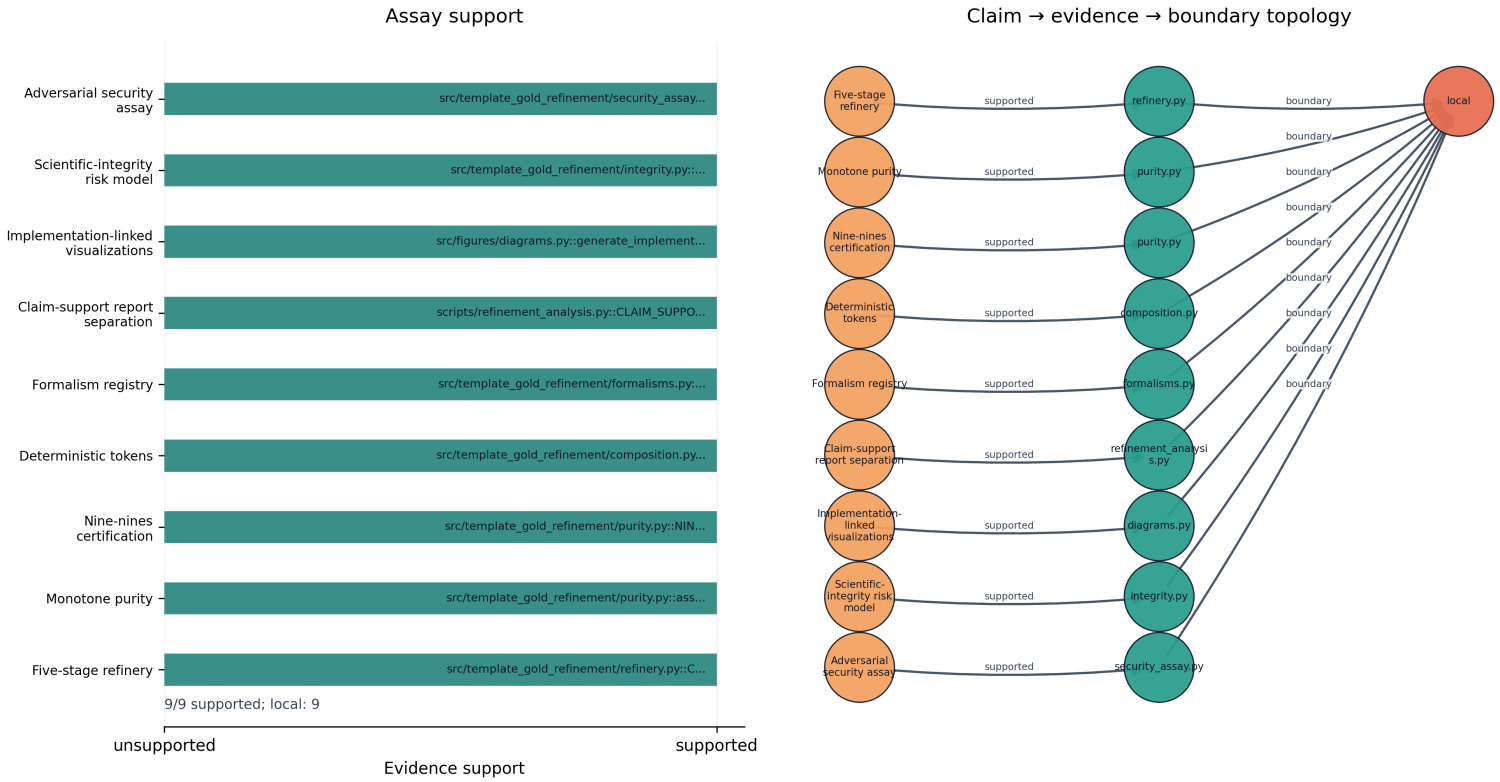


Figure 10: Claim-evidence assay showing supported contribution claims, evidence surfaces, and boundary classifications.

5.12 Scientific-integrity risk matrix

The integrity risk matrix in fig. 11 plots severity against detectability for the 9 integrity dimensions in tbl. 2. Bubble size encodes residual risk, while color encodes the source tier that owns the evidence surface. This makes boundary failures more visible than cosmetic source checks without hiding whether the support comes from config, source code, claim ledger, generated metric, artifact, bibliography, or validation gate. The matrix is intentionally local: it prioritizes where this exemplar needs source ownership, not where every future manuscript should focus.

The generated summary is: 9 integrity dimensions; highest residual risk is I4 (Analogy boundary) at 15. The matrix turns that summary into a triage surface. Severity asks how damaging a failure would be if it entered the manuscript. Detectability asks how readily the current project would catch it. Residual risk then becomes visible as size rather than being buried in a paragraph. A large point in a hard-to-detect region is a signal that the authoring contract needs stronger source ownership or a clearer boundary, even if the current text renders successfully.

Color is equally important because not all evidence tiers have the same authority. A bibliography-backed statement, a source-code computation, a claim ledger row, and a validation-gate result can all support prose, but they support different kinds of claims. The risk matrix keeps that distinction visible. It does not collapse integrity into a single score, and it does not imply that every high-severity issue has already been eliminated. It shows which risks this exemplar has made inspectable and where future work would need to add stronger measurement before using stronger language.

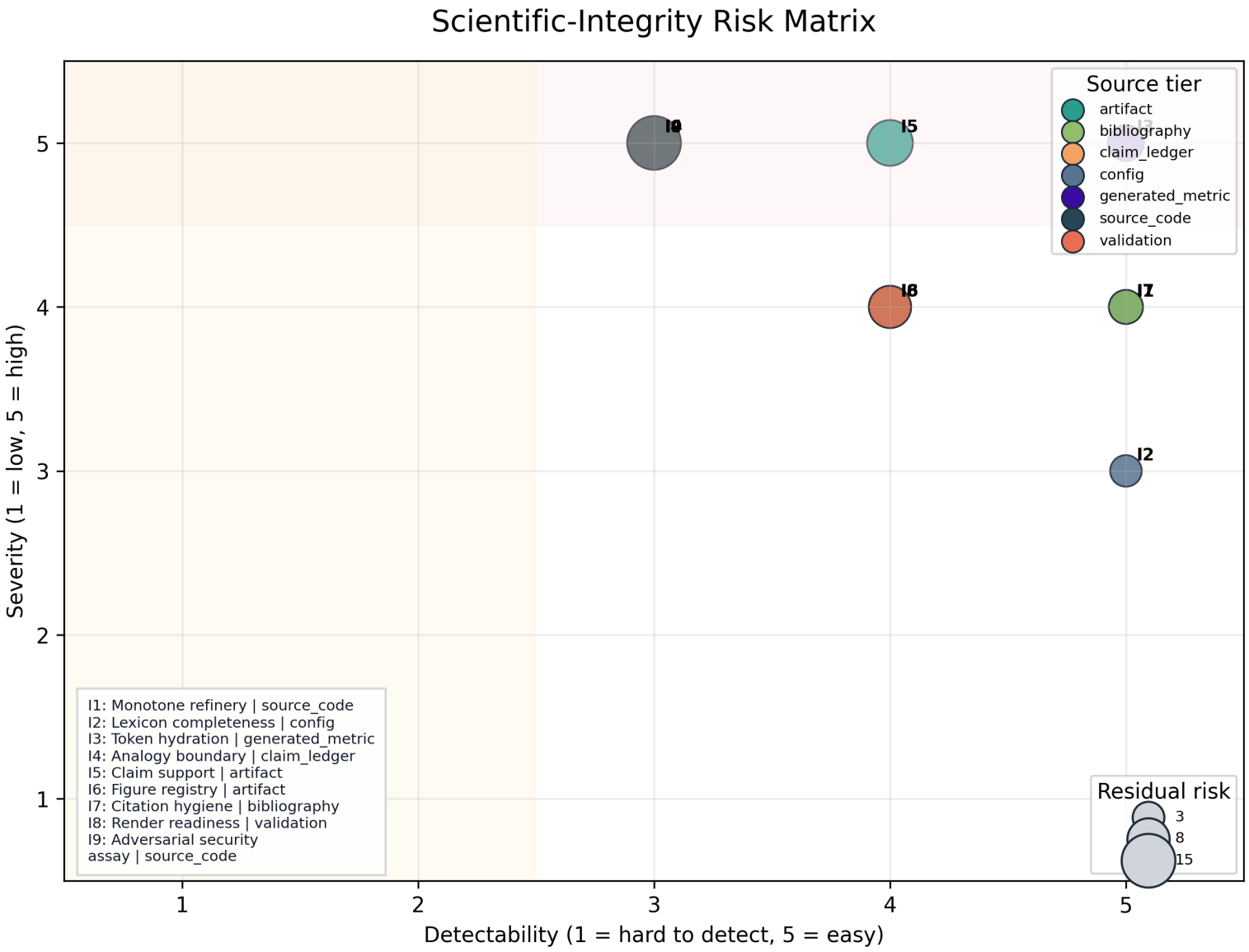


Figure 11: Scientific-integrity risk matrix plotting severity, detectability, residual risk, and owning evidence surface.

5.13 Evidence-tier ladder

The evidence-tier ladder in fig. 12 summarizes the evidence surfaces available to the shared template evidence registry or, before that gate has run, the fallback source tiers from the integrity model. It gives a quick view of whether the manuscript is leaning on generated metrics, claim-ledger facts, bibliography records, source code, or disposable artifacts.

The ladder complements the risk matrix by counting source tiers rather than plotting risks. When the shared evidence registry is available, the manuscript can report 960 source-tiered facts to the validation surface. When that registry is not available, the same figure falls back to the integrity model's configured tiers. Either way, the reader sees the evidentiary mix instead of receiving an undifferentiated assurance that evidence exists.

This matters because evidence balance is itself an integrity signal. A manuscript that leans only on generated artifacts may be reproducible but weakly grounded in source rationale. A manuscript that leans only on bibliography may be well-cited but not locally executable. A manuscript that leans only on tests may catch regressions while still failing to explain claim boundaries to readers. The ladder gives a compact audit of that mix, while tbl. 4 keeps the counts visible in tabular form. Together they close the figure sequence by showing not only that the 12 public figures render, but also which source tiers make their claims inspectable.

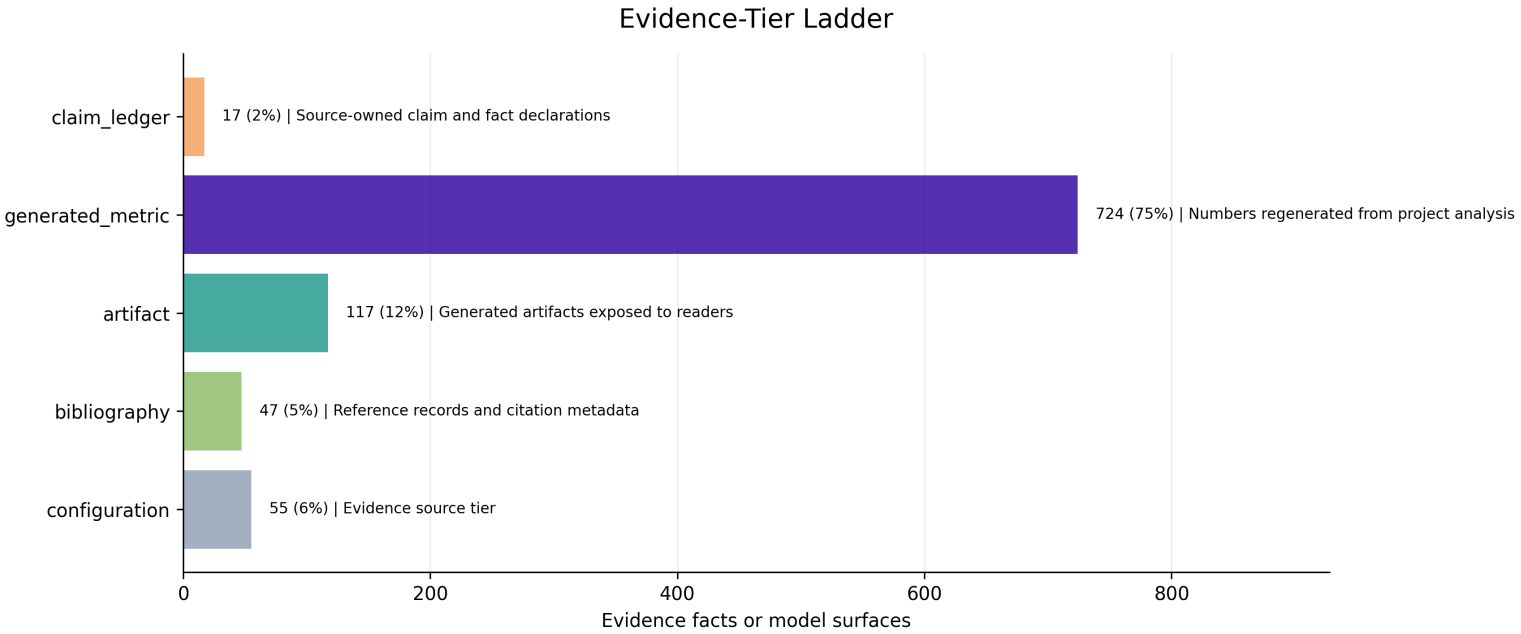


Figure 12: Evidence-tier ladder summarizing source tiers available to the shared template evidence registry.

Table 4: Evidence tiers used by the integrity model and shared registry when available.

Source tier	Count	Role
generated_metric	724	Numbers regenerated from project analysis
artifact	117	Generated artifacts exposed to readers
configuration	55	Evidence source tier
bibliography	47	Reference records and citation metadata
claim_ledger	17	Source-owned claim and fact declarations

5.14 Adversarial security assay

The adversarial assay reports 5 adversarial assay rows, 5 schema-complete, mapping threats and standards to local evidence surfaces, validators, and claim boundaries; completeness is a scope control, not completed scan findings. No Codex Security or Deep Security Scan findings are claimed unless a scan artifact is generated, validated, and cited. The rows are generated from `gold_refinement.security_assay` and are intentionally tabular rather than a new public figure, so the visual registry remains the stable 12-figure contract.

Table 5: Source-owned adversarial security assay.

ID	Threat	Standard or guidance	Evidence surface	Validator or gate	Claim boundary
S1	implicit trust in generated artifacts	NIST SP 800-207 zero trust	`output/reports/evidence_registry.json` and `src/template_gold_refinement/security_assay.py`	`infrastructure.validation.cli` evidence –fail-on-issues	documents a verification posture, not a deployed zero-trust architecture

ID	Threat	Standard or guidance	Evidence surface	Validator or gate	Claim boundary
S2	incomplete secure-development evidence	NIST SP 800-218 secure software development framework	`tests/`, pre-render validation, and claim ledger	project test suite and template validation gates	maps local practices to SSDF concepts without claiming SSDF compliance
S3	supply-chain or build provenance compromise	SLSA v1.2, Sigstore, SPDX, and CycloneDX	config hash, artifact counts, and publication metadata	pipeline regeneration and artifact registry checks	identifies provenance requirements but does not assert signed SBOM or provenance is present
S4	unvalidated vulnerability narrative	MITRE ATT&CK and Codex Security scan phases	security assay table and future scan artifacts	Codex Security threat-model, discovery, validation, and attack-path receipts when run	no real scan finding is claimed in this manuscript pass
S5	secure-by-design overclaim	CISA Secure by Design	authoring contract and scope limitations	human source review plus citation and evidence validation	uses guidance to bound responsibilities, not to certify product security

5.15 Contribution claims

Claim	Statement	Evidence	Boundary
Five-stage refinery	The refinery pipeline has 5 canonical stages from ore to nine-nines.	`src/template_gold_refinement/refinery.py::CANONICAL_STAGES`	local
Monotone purity	Purity increases strictly across all refinery stages.	`src/template_gold_refinement/purity.py::assert_monotone_increase`	local
Nine-nines certification	The certification stage achieves 99.9999999% purity.	`src/template_gold_refinement/purity.py::NINE_NINES_PURITY`	local
Deterministic tokens	Token selection is deterministic via seeded SHA-256 digest.	`src/template_gold_refinement/composition.py::_choose_value`	local
Formalism registry	The manuscript exposes 7 source-owned formalisms with equation labels.	`src/template_gold_refinement/formalisms.py::FORMALISMS`	local
Claim-support report separation	The project-local contribution-claim report is written to claim_support_registry.json.	`scripts/refinement_analysis.py::CLAIM_SUPPORT_REGISTRY_NAME`	local
Implementation-linked visualizations	The manuscript includes generated visualizations that link the refinery analogy to source code, variables, evidence, and validation gates.	`src/figures/diagrams.py::generate_implementation_circuit`	local
Scientific-integrity risk model	The manuscript includes a source-owned integrity risk model linking failure modes, validators, evidence surfaces, and fork obligations.	`src/template_gold_refinement/integrity.py::build_integrity_dimensions`	local
Adversarial security assay	The manuscript includes a source-owned security assay mapping adversarial threats and standards to local evidence surfaces, validators, and claim boundaries.	`src/template_gold_refinement/security_assay.py::build_security_assay`	local

The project-local claim-support assay reports 9 supported claims out of 9 total claims, for 100.00% support. Unsupported claims: 0.

The generated project report path is `output/reports/claim_support_registry.json`; the shared template evidence report remains `output/reports/evidence_registry.json`.

5.16 Shared evidence registry summary

When the template evidence gate has run, the shared registry supplies source-tiered facts used by the evidence validator. Current fact count available to this variable pass: 960.

Table 6: Shared evidence-registry fact kinds when available.

Fact kind	Count
artifact	73
citation	47
equation	8
figure	28
number	787
section	10
table	7

5.17 Figure quality report

The visualization registry is paired with `output/reports/figure_quality_report.json`, a generated QA report that checks PNG and SVG existence, file dimensions, nonblank pixel mass, color variance, and registry parity. Current status: passing with 12/12 registered figures passing and registry parity reported as Yes. PNG remains the manuscript render path; SVG is the companion technical artifact for inspection, reuse, and source-level debugging. tbl. 7 summarizes the generated surface.

Table 7: Figure-quality report generated from source-owned figure specs.

Figure	PNG	SVG	Dimensions	Nonwhite	Variance	Status
claim_evidence_assay	yes	yes	3952x2038	0.217	0.06049260	pass
evidence_tier_ladder	yes	yes	3430x1448	0.113	0.04049412	pass
formalism_traceability	yes	yes	3315x1797	0.140	0.04434550	pass
implementation_circuit	yes	yes	2966x1843	0.068	0.02205485	pass
integrity_gate_matrix	yes	yes	1818x2060	0.409	0.16204936	pass
integrity_risk_matrix	yes	yes	2499x1910	0.380	0.01852528	pass
karat_grading	yes	yes	2961x1698	0.279	0.07300982	pass
provenance_sankey	yes	yes	2850x1461	0.070	0.02178809	pass
purity_claim_scatter	yes	yes	2336x1744	0.034	0.01607749	pass
purity_progression	yes	yes	3024x2125	0.182	0.03967814	pass
token_density	yes	yes	3289x1856	0.234	0.06699486	pass
token_heatmap	yes	yes	2397x2399	0.627	0.12908659	pass

6 Discussion: Load-Bearing vs Rhetorical Analogy

6.1 Load-bearing vs rhetorical

The gold-refining analogy operates on two levels. **Rhetorically**, it provides a memorable framing for a methods paper: purity progression, karat grading, and certification are vivid metaphors for manuscript quality. **Operationally**, each stage maps to a real template-infrastructure operation - smelting to claim removal, assaying to evidence validation, cupellation to cross-reference resolution, and certification to full pipeline validation. The scholarship makes the test sharper: the analogy is useful only where the relation among stages is preserved, not where surface language makes writing sound like metallurgy [Gentner, 1983, Hesse, 1966].

The pre-1800 metallurgy sources strengthen the analogy by making that relation historically concrete, but they also narrow it. They show several traditions of staged work - ancient extraction and testing, Renaissance printed metallurgy, seventeenth-century goldsmith standards, and eighteenth-century assay manuals - rather than one timeless method [Pliny the Elder, c. 77 CE, Biringuccio, 1540, Agricola, 1556, W. B. (William Badcock), 1678, Cramer, 1741]. The manuscript therefore imports structure, not authority: cupellation does not make citation review chemical, a hallmark does not make a rendered PDF legally certified, and a karat scale does not make local validation equivalent to external truth.

The analogy is smelting the manuscript: it performs the refinement it describes. Its fork obligation is equally important. Gold refining can model staged purification, but it cannot certify domain truth unless the fork supplies a real domain validator and evidence source.

The added implementation and claim-assay figures sharpen that boundary. They show that "purity" is not a free-standing aesthetic judgment. It is a local statement about whether source-owned stages, generated artifacts, claim ledgers, figure registries, and validation commands agree. The figures therefore make a negative claim as important as the positive one: when a fork lacks a validator, a ledger, or a source artifact, the metaphor must stop at analogy and cannot be promoted to certification. That boundary follows reproducibility scholarship as much as metaphor theory: formal traceability can support rerunning and auditing, but it cannot by itself establish external validity [Peng, 2011, Wilkinson et al., 2016, Ioannidis, 2005].

The same caution applies to checklist-shaped infrastructure. Reporting guidelines improve the inspectability of research reports by naming information that should be present, but checklist completion is not the same as methodological quality, absence of bias, or truth of results [EQUATOR Network, 2026, Schulz et al., 2010, Page et al., 2021]. The gold-refinement gates should therefore be read as completeness and traceability checks. They can expose a missing citation, an unresolved token, or an unsupported local claim; they cannot convert weak domain evidence into strong domain evidence.

6.2 Useful adaptation cases

- **Domain-specific refinement pipelines**: fork the exemplar and remap stages to domain operations (e.g., clinical evidence, legal citation, engineering specification).
- **Staged-state visualization**: reuse the purity and karat vocabulary only when the fork declares what each state means, enforces ordering and continuity, and avoids presenting designed values as validated quality measurements.
- **Mega-madlib composition**: reuse the deterministic token engine for any config-owned lexical composition task.
- **Domain adapters**: use `src/template_gold_refinement/domain_adapter.py` and `domain_profile.yaml` to translate a domain's own metrics into the same purity scale before reusing certification language.
- **Research compendia**: package manuscript shells, token rules, analysis outputs, figures, and validation reports as a single reproducible object rather than a loose bundle of supplementary files [Marwick et al., 2018].

6.3 Misuse modes

Mode	Risk	Detection	Mitigation
Non-monotone purity	A stage has lower output purity than input.	`assert_monotone_increase` raises `ValueError`.	Fix stage purity targets in `src/template_gold_refinement/refinery.py`.
Empty lexicon category	A required lexicon category is empty or missing.	Config validation raises `GoldRefinementConfigError`.	Add vocabulary to `manuscript/config.yaml`.
Unresolved token	A manuscript placeholder has no generated variable.	`test_all_manuscript_tokens_are_generated` fails.	Add variable in `src/template_gold_refinement/manuscript_variables.py`.
Rhetorical-only analogy	The analogy is decorative with no operational mapping.	Review that each stage maps to a real infrastructure operation.	Connect stages to template pipeline operations.

Mode	Risk	Detection	Mitigation
Undetected integrity gap	A high-severity failure mode is present but no owner, validator, or generated artifact makes it visible.	`build_integrity_dimensions` lists severity, detectability, owner, validator, and evidence surface.	Add or revise the source-owned integrity dimension before promoting the manuscript.
Citation laundering	A real citation is used to make a stronger claim than the source supports.	Scope and evaluation prose separate analogy support, reproducibility support, and domain-evidence support.	Lower the claim boundary or add a source-owned validator before using certification language.
Checklist theater	A fork treats filled reporting rows or passed template gates as proof of scientific validity.	Methods, scope, discussion, and evaluation prose separate completeness, traceability, and reproducibility from domain truth.	Add the field's reporting guideline, domain validators, and independent evidence before making compliance or validity claims.
Executable-package ambiguity	A fork publishes a rendered manuscript without enough software, metadata, or version identity to rebuild the object.	Reproducibility, scope, evaluation, and authoring-contract prose require source-owned metadata, exact release citation, and regenerated output reports.	Record `software/template` release identity, preserve source-owned metadata, and rerun the full pipeline before publishing.
Security theater	Security language is presented as compliance, secure-by-design proof, or scan evidence without generated artifacts.	Security assay rows require a threat, standard, evidence surface, validator, and claim boundary.	Keep security claims bounded to the configured assay unless validated scan receipts exist.
Scan result laundering	A future scan summary is imported as prose without the artifact, receipt, and validator that produced it.	Authoring-contract and evaluation prose require scan artifacts before real Codex Security or Deep Security Scan findings are claimed.	Integrate only generated scan reports with evidence-registry alignment and explicit claim boundaries.

6.4 Design principles

Principle	Rationale
Analogy is load-bearing	Each metallurgical stage maps to a real template-infrastructure operation, not mere decoration.
Purity increases monotonically	The refinery pipeline guarantees strictly increasing purity from ore to certification.
Token selection is deterministic	A fixed seed and lexicon produce the same injection plan across reruns.
Configuration owns prose choices	Reviewers can inspect the declared language surface before generation.
Generated output is disposable	The durable artifact is the regeneration contract, not hand-edited output.
Scholarship sets boundaries	External sources can anchor analogy, reproducibility, and provenance claims, but they cannot expand local certification beyond what the source-owned evidence supports.
Completeness is not validity	Checklist-shaped gates can show that required reporting surfaces are present and traceable, but they do not certify domain truth, study design quality, or absence of bias.
Executable package is the object	Narrative, config, code, metadata, figures, reports, and rendered outputs must travel as a rebuildable package rather than as detached static prose.
Adversarial purity is bounded	Security standards frame threats and evidence requirements, but local certification cannot claim compliance or scan results without source-owned artifacts.

6.5 Analogy-break boundary

The analogy breaks when purity becomes a rhetorical grade detached from evidence. In this exemplar, eq. 4 and eq. 5 keep the grade tied to claim support and gate coverage. A fork that cannot provide comparable source-owned gates should keep the gold-refining language as metaphor only and avoid publication-strength claims.

The reverse assay clarifies a second boundary. It can identify the shortest prefix that reaches a **declared** target, but it cannot identify the cheapest, fairest, or scientifically best workflow unless stages have empirically justified costs and outcomes. Likewise, the multi-objective purity vector prevents compensatory averaging, but its dimensions are still local engineering observables rather than a validated psychometric scale.

The practical rule is simple: do not add an impressive figure unless its path through fig. 9 is visible. A visual can summarize an idea, but it only supports a manuscript claim when the claim-evidence assay can point to the owning file, symbol, generated artifact, and validation surface.

The integrity risk matrix adds one more brake. A fork may have all figures registered and still be scientifically weak if its highest-severity risks are only weakly detectable. In this exemplar, tbl. 2 makes that weakness inspectable by pairing each dimension with an owner and validator. The useful question for a fork is therefore not "can the manuscript render?" but "which severe failures would the current pipeline miss, and what source-owned gate would expose them?"

The evidence-tier ladder also prevents a common drift in generated manuscripts: treating generated artifacts as if they were independent evidence. A generated metric can support an internal consistency claim, but a domain claim needs a domain source tier. The ladder makes that distinction visible without pretending that a large fact count is the same as stronger external validity. In FAIR and PROV terms, richer metadata improves findability, reuse, and traceability, but it does not convert weak evidence into strong evidence [Wilkinson et al., 2016, Moreau et al., 2013].

Open-science standards push in the same direction. The TOP Guidelines and UNESCO Recommendation on Open Science emphasize transparency, scrutiny, reproducibility, accountability, and shared research objects [Nosek et al., 2015, UNESCO, 2021]. This exemplar implements a small local version of those values; it does not claim that openness alone resolves study design, fairness, or domain-validity problems.

Security guidance adds an adversarial version of the same brake. Zero trust, secure development, supply-chain provenance, signing, SBOM, attack-path, and secure-by-design frameworks are useful because they ask who or what should not be trusted by default [National Institute of Standards and Technology, 2020, 2022, Open Source Security Foundation, 2026, Sigstore, 2026, MITRE, 2026, CycloneDX, 2026, SPDX, 2026, Cybersecurity and Infrastructure Security Agency, 2026]. In this manuscript they do not certify security. They make overclaiming harder: a security claim must point to tbl. 5, and a real scan claim must point to scan artifacts that this pass does not produce.

7 Conclusion: Certification and Forking

The gold-refinery pipeline demonstrates that a metallurgical analogy can be made operationally accountable: each stage maps to an executable transformation, stage order and continuity are enforced, artifacts preserve provenance, and validators constrain the terminal claim. The canonical run reaches the local nine-nines state (99.9999999% (nine-nines)); this is a software predicate, not a universal certification of manuscript quality.

7.1 Summary

- 5 refinery stages from ore (9K) to certification (nine-nines)
- Final purity: 99.9999999% (nine-nines) (24K (nine-nines certified))
- 24 tokens generated deterministically from seed 431
- Config hash: f07f54804db53e6d
- 7 source-owned formalisms with equation labels: eq:purity_functional, eq:monotone_refinery, eq:token_digest, eq:claim_support, eq:integrity_vector, eq:certification_predicate, eq:adversarial_assay
- Claim-support status: 9/9 supported (passing)
- 9 integrity dimensions with residual-risk scoring and owner/validator links.
- Registered visual evidence spans purity progression, token coverage, formalism traceability, implementation flow, claim-evidence assay, risk-matrix, and evidence-tier surfaces.
- Reverse assay returns the shortest ordered stage prefix for a declared target, and multi-objective purity preserves distinct quality dimensions without compensatory averaging.

7.2 Forking responsibilities

1. Remap metallurgical stages to domain operations
2. Update lexicon categories in `manuscript/config.yaml`
3. Add domain-specific evidence and validators
4. Regenerate all outputs through the pipeline
5. Do not hand-edit generated manuscript, PDFs, or figures

The durable result is not the current prose snapshot. It is the reproducible contract that can rebuild the prose, figures, formalisms, and validation reports from source.

That contract is the paper's central contribution. It demonstrates how an analogy can be useful without being loose: every metaphorical move either points to a source-owned implementation surface or stays explicitly inside the discussion boundary. The paper therefore contributes a reproducible-composition pattern, not a universal theory of manuscript quality.

The added integrity model makes the same claim in a stricter form: every high-value visual or formal statement should have an owner, an evidence tier, and a validator. Without those three surfaces, the right outcome is not a more polished metaphor; it is a narrower claim. This keeps the final certification claim aligned with reproducible-research norms: the artifact is stronger when it can be regenerated and audited, but still bounded by the evidence its sources actually contain [Sandve et al., 2013, Marwick et al., 2018].

8 Reproducibility: Seeded Regeneration

Reproduction requires identity, execution, and verification—not merely access to a PDF. Identity fixes the source revision, configuration hash, software release, and environment. Execution rebuilds analysis artifacts before manuscript hydration. Verification checks that registered claims, figures, references, and rendered formats agree with their source owners.

8.1 Deterministic regeneration

The refinery pipeline is fully deterministic. Given the same `manuscript/config.yaml` and `src/` code, every run produces identical output. This is the local version of a reproducible computational research norm: the reader should be able to inspect the source, rerun the workflow, and recover the same derived artifacts [Peng, 2011, Sandve et al., 2013].

Executable-publication scholarship sharpens that norm. Executable research compendia and executable papers treat an article as a package of narrative, code, data, environment, and rendered outputs rather than as a static document with detachable supplements [Nüst et al., 2017, Lasser, 2020]. The present exemplar is smaller and more template-specific: it does not provide a universal executable-paper format, but it does make the manuscript variables, figures, reports, and rendered PDF/HTML products rebuildable from source-owned inputs.

- **Seed:** 431
- **Config hash:** f07f54804db53e6d
- **Generation timestamp:** 2026-10-15T18:40:52Z
- **Python version:** 3.14.4

8.2 Artifact inventory

Category	Count
Figures	26
Data files	3
Reports	17
Total	46

8.3 Regeneration commands

```
# Run the refinery analysis
uv run python projects/templates/template_gold_refinement/scripts/refinement_analysis.py

# Generate manuscript variables
uv run python projects/templates/template_gold_refinement/scripts/z_generate_manuscript_variables.py

# Full pipeline (from repo root)
./run.sh --project templates/template_gold_refinement --pipeline --mode core
```

A reproduction report should record command exit status, the source revision, `f07f54804db53e6d`, Python 3.14.4, and whether the generated registries pass. Matching prose alone is insufficient if the token plan, claim registry, or figure registry differs. Conversely, timestamp or renderer metadata differences should be interpreted separately from substantive differences in source-owned values.

8.4 Config ownership

All vocabulary, slots, section conditions, steganography toggles, and optional LLM review gates are declared in `manuscript/config.yaml` under `gold_refinement:`, `steganography:`, and `llm:`. The config is the source of truth; generated prose is disposable.

`src/template_gold_refinement/pipeline_policy.py` turns those policy blocks into an explicit secure-pipeline hook. That keeps the optional hardening path visible before execution instead of burying it in shell glue or prose.

The reproducibility spine uses fact registry and figure registry as generated artifacts rather than reader trust signals. Variable generation records `f07f54804db53e6d`; analysis writes refinery, token, claim-support, dashboard, and figure artifacts; validation may add the shared evidence registry used by template scientific-integrity checks.

The implementation circuit gives a reproducibility checklist for future forks. A reader should be able to start at any rendered figure or claim, follow it to a generated variable or report, follow that artifact to `src/` or `manuscript/config.yaml`, and rerun the same stage command. If that path is broken, the fork has produced a static illustration rather than a reproducible refinement pipeline.

Metadata is part of that path, not administrative garnish. Work on reproducible computational metadata argues that data, tools, workflows, environments, and reports need machine-readable descriptors before re-execution and reuse can be automated reliably [Chen et al., 2021]. Here, the evidence registry, token plan, figure registry, output statistics, and config hash are the local metadata

stack. They make the object inspectable, but they remain descriptive: they identify what was generated and from where, not whether a domain conclusion is correct.

The same rule applies to visual polish. The figure registry is source-owned, every registered PNG now has a companion SVG, and `output/reports/figure_quality_report.json` records 12 PNG files, 12 SVG files, and 12 passing visual-quality checks for the current variable pass. A fork should treat tbl. 7 as the figure-layer analogue of the claim-support registry: it proves that the visuals are regenerated, present in both raster and vector forms, nonblank, and aligned with the registry before prose promotes them.

8.5 Evidence-registry separation

This exemplar now separates two evidence surfaces. `output/reports/claim_support_registry.json` is the project-local contribution-claim assay consumed by the dashboard and claim-support figure. `output/reports/evidence_registry.json` is reserved for the shared template evidence validator, which registers numbers, citations, equations, sections, figures, tables, generated data, and claim-ledger facts. The split mirrors provenance standards: different entities can be generated by different activities and still belong to one accountable research object [Moreau et al., 2013, Belhajjame et al., 2015].

The evidence-tier ladder in fig. 12 is the reproducibility counterpart to that separation. It does not merely count files. It shows which tier owns each support surface, so a future fork can see whether a conclusion rests on config declarations, generated metrics, claim-ledger facts, bibliography records, or rendered artifacts. That distinction matters because only some tiers can support domain truth; others support reproducibility, formatting, or local consistency.

9 Scope: Related Work and Limitations

9.1 Scope limitations

This exemplar demonstrates the gold-refining analogy as a **methods paper**. It does not claim:

- Empirical validation of manuscript quality metrics against external standards
- Generalizability of specific purity fractions to all scientific domains
- That the analogy replaces domain-specific peer review or expert judgement
- That reverse assay optimizes cost or scientific value, or that multi-objective purity is a validated composite scale

9.2 Related work

The mega-madlib token injection pattern follows `template_madlib`'s deterministic lexical composition approach [Friedman, 2026a]. The pipeline-staging model draws on `template_code_project`'s thin-orchestrator pattern and the wider template repository's validation and rendering infrastructure [Friedman, 2026b]. The refinement analogy is novel to this exemplar, but the surrounding scholarship is not.

Analogy theory supplies the first boundary. Structure-mapping treats analogy as a transfer of relational organization rather than surface resemblance [Gentner, 1983], while philosophy of science distinguishes positive, negative, and still-open analogy regions [Hesse, 1966]. The paper therefore claims that the refinery stages organize a reproducible manuscript workflow. It does not claim that metallurgical purity is an empirical measure of prose quality.

Reproducible-research scholarship supplies the second boundary. Literate programming, Sweave, and notebook-based analysis show that code, results, and narrative can be co-developed in executable documents [Knuth, 1984, Leisch, 2002, Rule et al., 2019]. Research compendia and workflow-centric research objects show how code, data, text, environment, and provenance can be packaged as durable units [Marwick et al., 2018, Belhajjame et al., 2015]. This exemplar extends those ideas to deterministic token selection, but it remains an internal consistency and provenance demonstration.

FAIR and PROV supply the third boundary. Rich metadata and provenance improve findability, interoperability, reuse, and auditability [Wilkinson et al., 2016, Moreau et al., 2013]. They do not guarantee that a substantive scientific claim is true. The evidence ladder in this paper is therefore an honesty device: source-code facts, generated metrics, bibliography records, and domain evidence must not be collapsed into one undifferentiated support score.

The metallurgy literature supplies the fourth boundary. This pass grounds the source domain primarily in pre-1800 texts: Pliny for metals, extraction, and touchstone testing; Biringuccio and Agricola for printed descriptions of mining, smelting, parting, and assay; Badcock's *Touch-stone* and the Goldsmiths' Company chronology for standards, statutes, and marks; and Cramer for eighteenth-century assay as a theory/practice discipline [Pliny the Elder, c. 77 CE, Biringuccio, 1540, Agricola, 1556, W. B. (William Badcock), 1678, The Goldsmiths' Company Assay Office, 2026, Cramer, 1741]. Modern gold-extraction and hallmarking sources remain useful as contrast and current vocabulary [Marsden and House, 2006, Hallmarking Convention, 1972, London Bullion Market Association, 2026]. This paper imports relational structure. It does not import market rules, assay tolerances, local guild authority, mint authority, or regulatory power into manuscript review.

Structured reporting and open-science standards supply the fifth boundary. CONSORT, STROBE, PRISMA, ARRIVE, and EQUATOR show how research communities turn recurring omissions into explicit reporting items [Schulz et al., 2010, von Elm et al., 2007, Page et al., 2021, Percie du Sert et al., 2020, EQUATOR Network, 2026]. The TOP Guidelines and UNESCO Recommendation on Open Science broaden that logic toward transparency, sharing, accountability, and reproducibility [Nosek et al., 2015, UNESCO, 2021]. This exemplar is guideline-shaped, but it is not a replacement for discipline-specific reporting compliance: forks must choose the appropriate external checklist and add domain validators before claiming compliance with any field standard.

Executable-publication and software-citation work supply the sixth boundary. Executable research compendia, executable papers, and analytic-stack metadata show how publication objects can bind narrative, code, data, environments, and outputs into a reusable package [Nüst et al., 2017, Lasser, 2020, Chen et al., 2021]. Software-citation principles add that code and template releases should be credited with enough specificity, persistence, and accessibility that readers can identify the version used [Smith et al., 2016]. This exemplar aligns with those norms, but it does not claim that a regenerated PDF is itself an archival preservation system or that a repository URL substitutes for versioned software citation.

Security and supply-chain guidance supply the seventh boundary. Zero-trust architecture, SSDF, SLSA, Sigstore, MITRE ATT&CK, CycloneDX, SPDX, and Secure by Design each name useful threat or provenance surfaces [National Institute of Standards and Technology, 2020, 2022, Open Source Security Foundation, 2026, Sigstore, 2026, MITRE, 2026, CycloneDX, 2026, SPDX, 2026, Cybersecurity and Infrastructure Security Agency, 2026]. This exemplar uses those sources to structure an adversarial assay. It does not claim compliance with those standards, it does not claim a signed SBOM or provenance bundle, and it does not claim a Codex Security or Deep Security Scan has been run.

9.3 Responsible forking

A fork must:

1. Add domain-specific evidence before making domain claims
2. Update lexicon categories to reflect domain vocabulary
3. Connect refinery stages to real domain operations
4. Add domain validators beyond the exemplar's generic gates
5. Use `src/template_gold_refinement/domain_adapter.py` and `docs/domain_fork_guide.md` to remap domain metrics and boundary notes
6. Cite the exact software/template release and environment used for the fork
7. Update the adversarial security assay when the fork changes threat scope or supply-chain evidence
8. Regenerate all outputs through the pipeline

The formalism registry is local to this exemplar. It states how this project maps refinery stages, token selection, and evidence gates into equations; it does not prove that gold refining is a universal model of scientific writing. Domain forks should replace or narrow the formalism set before reusing the certification language.

The same limitation applies to the integrity risk model. The current 9 dimensions are tuned to a template exemplar: token hydration, figure registration, claim support, citation hygiene, render readiness, and analogy boundaries. A fork that studies a real domain must add domain-specific risks, domain evidence tiers, and validators before treating fig. 11 as a publication-readiness claim.

The adversarial assay is similarly local. It records 5 rows that make security scope inspectable, but it is not a security attestation. A fork that wants secure-by-design, zero-trust, supply-chain, or vulnerability-scan claims must add the missing artifacts, validators, receipts, and external review before reusing that language.

The software improvements in this version do not remove these limitations. Enforced continuity proves that the configured stages connect; reverse assay proves only that a prefix reaches a declared numerical target; and `PurityVector` prevents accidental compensatory averaging. None establishes that the chosen targets, dimensions, or stage meanings are externally valid.

10 Quality Probes

10.1 QA probes

Probe	Question	Passing signal	Artifact
Monotone purity	Does purity increase strictly across all refinery stages?	`assert_monotone_increase` passes on the purity sequence.	`src/template_gold_refinement/refinery.py` and `output/data/refinery_results.json`
Token provenance	Can every selected token be traced to a category, section, value, and config key?	The token plan contains one row for each generated token.	`output/reports/token_plan.json`
Karat grade correctness	Does each stage map to the correct karat grade?	`karat_for_purity` returns the expected grade for each stage.	`src/template_gold_refinement/purity.py`
Integrity risk visibility	Can the manuscript identify high-severity integrity failures and the validator or artifact that detects them?	The integrity risk model emits dimensions, owners, residual risk scores, and evidence-tier rows.	`src/template_gold_refinement/integrity.py` and `../figures/integrity_risk_matrix.png`
Scholarship boundary	Do pre-1800 metallurgy references support the analogy, reproducibility, provenance, and source-domain framing without being used as evidence for universal manuscript-quality claims?	The scope, discussion, and evaluation sections cite historically bounded metallurgy scholarship while keeping certification local to source-owned gates.	`manuscript/references.bib` and `manuscript/07_scope.md`
Reporting-guideline completeness	Does the manuscript distinguish checklist-style completeness from methodological validity?	The methods, scope, discussion, and evaluation sections cite reporting-guideline scholarship while explicitly limiting what the local gates prove.	`manuscript/02_methodology.md`, `manuscript/04_discussion.md`, `manuscript/07_scope.md`, and `manuscript/08_evaluation.md`
Executable-compendium identity	Can a reader identify the executable package, metadata stack, software release, and generated artifacts needed to rebuild the manuscript?	The reproducibility, scope, evaluation, and authoring-contract sections cite executable-publication and software-citation scholarship while keeping preservation and portability claims bounded.	`manuscript/06_reproducibility.md`, `manuscript/07_scope.md`, `manuscript/08_evaluation.md`, `manuscript/09_authoring_contract.md`, `output/reports/evidence_registry.json`, and `output/reports/output_statistics.json`
Adversarial assay boundary	Does security language distinguish declared threat scope from real scan evidence and external compliance?	The security assay emits threats, standards, evidence surfaces, validators, and claim boundaries without claiming Codex Security findings.	`src/template_gold_refinement/security_assay.py`, `manuscript/config.yaml`, and `manuscript/03_results.md`

The selected evaluation gate terms are prerender and citation validation. They are intentionally narrower than peer review: they check source ownership, token coverage, figure registration, claim support, and rendering integrity before a human reviewer assesses the substantive analogy.

Evaluation uses three noninterchangeable levels. **Invariant tests** reject invalid executable states such as nonmonotone, discontinuous, or misordered stages. **Artifact checks** establish presence, readability, provenance, and registry parity. **Claim-boundary review** asks whether the prose says no more than those tests and artifacts support. Certification requires all applicable levels; a pass at one level cannot compensate for failure at another.

10.2 Audit rules

Rule	Check	Test
Purity monotonicity	Purity must strictly increase from stage to stage	tests/test_refinery.py

Rule	Check	Test
Token determinism	Same seed and lexicon must produce same token plan	tests/test_composition.py
Token coverage	Every manuscript `{{TOKEN}}` must have a generated variable	tests/test_manuscript_variables.py
Config validation	Invalid config must raise GoldRefinementConfigError	tests/test_config.py
Figure generation	All figure generators must produce non-blank PNGs	tests/test_figures.py
Integrity model coverage	Integrity dimensions must have unique IDs, owners, validators, and evidence surfaces	tests/test_integrity.py
Scholarship boundary	Pre-1800 metallurgy citations must anchor framing and limitations without substituting for domain validation	prerender citation/evidence validation plus human source review
Reporting-guideline boundary	Checklist-style completeness must not be represented as methodological validity or external reporting compliance	prerender citation/evidence validation plus human source review
AI/template accountability	Tool assistance must not be represented as authorship or independent responsibility	authoring-contract source review plus publication-ethics citation check
Executable-package identity	The manuscript must distinguish a regenerated local package from long-term archival preservation or universal executable-paper compliance	reproducibility source review plus output statistics and evidence-registry validation
Security assay boundary	Security standards and scan phases must be represented as scoped guidance unless generated scan artifacts exist	tests/test_security_assay.py and manuscript source review

The audit rules are summarized visually in fig. 7 and algebraically in eq. 5. A failed audit rule should block certification language even if the PDF renders.

Scholarship adds one more gate: citation validity and claim-boundary discipline. A source can support a relation, a practice, or a caution without supporting every attractive extrapolation from that source. The evaluation surface therefore treats analogy theory, reproducibility literature, provenance standards, and pre-1800 metallurgy references as boundary-setting evidence, not as decorations added after the pipeline already decided the claim [Gentner, 1983, Peng, 2011, Wilkinson et al., 2016, Biringuccio, 1540, Agricola, 1556, W. B. (William Badcock), 1678, Cramer, 1741]. A passing source review must preserve the distinction between historically documented assay/marking practices and this exemplar's local software certification predicate.

The reporting-guideline literature keeps that gate modest. A passed checklist row certifies that a required reporting surface is present and traceable; it does not certify that the study behind the report is unbiased, sufficiently powered, or externally valid [EQUATOR Network, 2026, von Elm et al., 2007, Percie du Sert et al., 2020]. The same rule governs the quality probes here. Passing them supports local claims about source ownership, artifact completeness, and reproducible rendering, not universal claims about manuscript quality.

Executable-compendium scholarship adds a more operational evaluation target: can a reader identify the paper object, its source code, its generated artifacts, its metadata, and the software version needed to rebuild it [Nüst et al., 2017, Chen et al., 2021, Smith et al., 2016]? The current gates answer that question locally through variable hydration, artifact counts, evidence facts, figure registration, and PDF/HTML validation. They do not measure long-term preservation, cross-platform portability, reviewer workload, or reader comprehension; those would require separate empirical studies.

The adversarial assay adds a security-specific evaluation target: can a reader distinguish declared threat scope from actual scan evidence? tbl. 5 maps zero-trust, secure-development, supply-chain, SBOM, attack-path, and secure-by-design guidance to local evidence surfaces and validators [National Institute of Standards and Technology, 2020, 2022, Open Source Security Foundation, 2026, Sigstore, 2026, MITRE, 2026, CycloneDX, 2026, SPDX, 2026, Cybersecurity and Infrastructure Security Agency, 2026]. The passing condition is bounded: the table must be complete and the prose must not claim compliance or Codex Security findings unless future scan artifacts are generated and integrated.

The risk model adds prioritization to the gate list. fig. 11 separates easy-to-detect implementation failures from severe boundary failures that need clearer ownership. This keeps the evaluation surface from becoming a checklist of equally weighted boxes: token coverage, citation validity, claim support, and render readiness all matter, but a high-severity low-detectability failure should shape the next source edit before cosmetic manuscript polish.

Figure review follows the same rule. File existence and nonblank pixels are necessary but not sufficient. The figure caption, encoding declaration, source data, and prose interpretation must describe the same measurement. This criterion specifically prohibits inferring a stagewise trend from a project-level aggregate, which is why fig. 5 repeats one observed claim-support rate rather than inventing cumulative values.

11 Authoring Contract

11.1 Obligations

Obligation	Requirement
Domain validator	Add domain-specific evidence before making domain claims beyond the exemplar.
Config ownership	Keep lexicon and slots in config.yaml, not in generated prose.
Regeneration contract	Regenerate outputs through the pipeline, not by hand-editing.
Risk review	Treat high-residual-risk integrity dimensions as fork obligations before publication claims are expanded.
AI and template disclosure	Disclose material AI, template, and automation assistance, and keep responsibility for accuracy and claim boundaries with named human authors.
Software and package citation	Cite the exact software/template release and preserve enough metadata for readers to identify and rebuild the executable manuscript object.
Security evidence boundary	Treat security standards and Codex Security scan phases as scoped guidance unless generated scan artifacts and receipts are present.

The authoring boundary tokens for this section are analogy boundary and non-claim. They mark the point where an author must either add new evidence and validators or lower the claim from certification to analogy.

Authorship remains accountable even when composition is deterministic. The token plan can explain which configured phrase entered which section, but it cannot take responsibility for whether the resulting claim is fair, cited, or appropriately bounded. Human authors must review the generated text against the source evidence, especially when the prose imports metaphorical force from metallurgy or borrows authority from reproducibility standards.

That accountability rule follows current publication-ethics guidance. ICMJE's 2026 Recommendations include a dedicated section on artificial intelligence in publishing, and COPE's authorship guidance states that AI tools cannot be listed as authors because they cannot accept responsibility for a manuscript [International Committee of Medical Journal Editors, 2026, Committee on Publication Ethics, 2023]. A deterministic template is different from a generative AI system, but the obligation is parallel: tooling may assist composition and validation, while named human authors remain responsible for disclosure, accuracy, evidence boundaries, and final claims.

Software citation closes the same accountability loop for executable materials. The code, template, and release that generated a manuscript should be treated as research products with credit, attribution, persistent identification, accessibility, and version specificity [Smith et al., 2016]. A fork should therefore cite the release it used and record enough metadata for readers to distinguish "same template family" from "same executable object."

Security authorship has the same rule. Standards and guidance can shape the threat model, but they cannot be cited as proof that this manuscript is compliant or secure. Future Codex Security or Deep Security Scan reports should enter the manuscript only as generated artifacts with validator receipts, evidence-ledger alignment, and a clear boundary between findings, mitigations, and remaining scope.

11.2 Fork checklist

1. Remap metallurgical stages to domain operations in `src/template_gold_refinement/refinery.py`
2. Update lexicon categories in `manuscript/config.yaml` under `gold_refinement.lexicon`
3. Update `contribution_claims` with domain-specific evidence pointers
4. Add domain validators beyond the exemplar's generic gates
5. Replace or extend `src/template_gold_refinement/integrity.py` dimensions when the fork introduces new failure modes
6. Update `domain_profile.yaml`, `src/template_gold_refinement/domain_adapter.py`, and `docs/domain_fork_guide.md` when the analogy is forked into a new domain
7. Keep `steganography` and `llm` policy blocks explicit when the secure pipeline or optional review path is used
8. Cite the exact template/software release and record environment metadata for the executable package
9. Update `gold_refinement.security_assay` and add scan artifacts before making secure-by-design, supply-chain, or vulnerability findings claims
10. Regenerate all outputs through the pipeline:

```
uv run python scripts/refinement_analysis.py
uv run python scripts/z_generate_manuscript_variables.py
```

11. Do not hand-edit generated manuscript, PDFs, or figures

12. If using reverse assay, preserve ordered-prefix semantics and disclose that the target is configured rather than empirically optimized
13. If using multi-objective purity, report dimensions separately unless external evidence justifies an aggregation rule

Do not hard-code equation, figure, or table numbers in prose. Use `[@eq:...]`, `[@fig:...]`, `[@tbl:...]`, and `[@sec:...]` so the renderer owns numbering and the tests can detect dangling references.

The authoring contract treats the risk matrix as a source checklist, not as a retrospective dashboard. If a fork cannot name who owns a high-severity integrity dimension, which validator detects it, and which evidence tier supports it, the fork should lower the claim boundary until that missing surface exists.

The same rule applies to the adversarial assay. If a fork cannot name the threat, standard, local evidence surface, validator, and claim boundary for a security claim, the fork should keep the language as scoped guidance rather than certification.

Visualization authorship is equally accountable. A technically valid image can still mislead through an unsupported axis, aggregation, annotation, or caption. Every figure change must therefore update its `FigureSpec`, source data declaration, tests, manuscript interpretation, and regenerated PNG/SVG pair as one reviewable unit.

12 Transmission ends

This source-owned bookend marks the end of the manuscript payload. A release must retain both bookends so a copied or truncated transmission cannot appear complete by filename alone.

References

Georgius Agricola. *De re metallica*. Hieronymus Froben and Nicolaus Episcopius, Basel, 1556. URL https://archive.org/details/deremetallica50agri. Hoover and Hoover English translation consulted via Internet Archive; source text first published in 1556.

Khalid Belhajjame, Oscar Corcho, Daniel Garijo, Jun Zhao, Paolo Missier, David Newman, Raul Palma, Sean Bechhofer, Esteban Garcia-Cuesta, Jose Manuel Gomez-Perez, et al. Using a suite of ontologies for preserving workflow-centric research objects. *Journal of Web Semantics*, 32:16–42, 2015. doi: 10.1016/j.websem.2015.01.003.

Vannoccio Biringuccio. *De la Pirotechnia*. Venturino Roffinello, Venice, 1540. URL https://digital.sciencehistory.org/works/n888ils. Public-domain first edition digitized by the Science History Institute.

Jonathan B. Buckheit and David L. Donoho. Wavelab and reproducible research. In Anestis Antoniadis and Georges Oppenheim, editors, *Wavelets and Statistics*, pages 55–81. Springer, New York, 1995. doi: 10.1007/978-1-4612-2544-7_5.

Xiaoli Chen et al. The role of metadata in reproducible computational research. *Patterns*, 2(9):100328, 2021. doi: 10.1016/j.patter.2021.100328.

Committee on Publication Ethics. Authorship and ai tools, 2023. URL https://publicationethics.org/cope-position-statements/ai-author.

Johann Andreas Cramer. *Elements of the Art of Assaying Metals*. T. Woodward, London, 1741. URL https://catalog.hathitrust.org/Record/001114669. HathiTrust catalogue record for the English edition.

Cybersecurity and Infrastructure Security Agency. Secure by design, 2026. URL https://www.cisa.gov/resources-tools/resources/secure-by-design. Accessed 2026-06-30.

CycloneDX. Cyclonedx specification overview, 2026. URL https://cyclonedx.org/specification/overview/. Accessed 2026-06-30.

EQUATOR Network. What is a reporting guideline?, 2026. URL https://www.equator-network.org/about-us/what-is-a-reporting-guideline/. Accessed 2026-06-30.

Daniel Ari Friedman. Template madlib exemplar. https://github.com/docxology/template/tree/main/projects/templates/template_madlib, 2026a.

Daniel Ari Friedman. Research project template. https://github.com/docxology/template, 2026b.

Dedre Gentner. Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, 7(2):155–170, 1983. doi: 10.1207/s15516709cog0702_3.

Hallmarking Convention. Convention on the control and marking of articles of precious metals, 1972. URL https://hallmarkingconvention.org/. Common Control Mark treaty framework, with later amendments and technical annexes.

Mary B. Hesse. *Models and Analogies in Science*. University of Notre Dame Press, Notre Dame, IN, 1966.

Keith J. Holyoak and Paul Thagard. *Mental Leaps: Analogy in Creative Thought*. MIT Press, Cambridge, MA, 1995.

International Committee of Medical Journal Editors. Recommendations for the conduct, reporting, editing, and publication of scholarly work in medical journals, 2026. URL https://www.icmje.org/recommendations/. Updated January 2026.

John P. A. Ioannidis. Why most published research findings are false. *PLOS Medicine*, 2(8):e124, 2005. doi: 10.1371/journal.pmed.0020124.

Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984. doi: 10.1093/comjnl/27.2.97.

Jana Lasser. Creating an executable paper is a journey through open science. *Communications Physics*, 3:224, 2020. doi: 10.1038/s42005-020-00403-4.

Friedrich Leisch. Sweave: Dynamic generation of statistical reports using literate data analysis. In *Proceedings of the 3rd International Workshop on Distributed Statistical Computing*, Vienna, Austria, 2002. URL https://www.statistik.tu-dortmund.de/useR-2008/slides/Leisch.pdf.

London Bullion Market Association. Good delivery rules for gold and silver bars, 2026. URL https://www.lbma.org.uk/good-delivery/good-delivery-rules. Current public rules consulted for fineness and certification framing.

John O. Marsden and C. Iain House. *The Chemistry of Gold Extraction*. Society for Mining, Metallurgy, and Exploration, Littleton, CO, 2 edition, 2006.

Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data analytical work reproducibly using r (and friends). *The American Statistician*, 72(1):80–90, 2018. doi: 10.1080/00031305.2017.1375986.

MITRE. Mitre att&ck, 2026. URL https://attack.mitre.org/. Accessed 2026-06-30.

Luc Moreau, Paul Groth, James Cheney, et al. Prov-overview: An overview of the prov family of documents. W3C Note, 2013. URL https://www.w3.org/TR/prov-overview/.

National Institute of Standards and Technology. Zero trust architecture. Technical Report Special Publication 800-207, National Institute of Standards and Technology, 2020. URL https://csrc.nist.gov/pubs/sp/800/207/final.

National Institute of Standards and Technology. Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities. Technical Report Special Publication 800-218, National Institute of Standards and Technology, 2022. URL https://csrc.nist.gov/pubs/sp/800/218/final.

Brian A. Nosek, George Alter, George C. Banks, Denny Borsboom, Sara D. Bowman, Steven J. Breckler, Stuart Buck, Christopher D. Chambers, Gilbert Chin, Garret Christensen, Maurizio Contestabile, Allan Dafoe, Eric Eich, Jeremy Freese, Rachel Glennerster, Daniel Goroff, Donald P. Green, Brian Hesse, Macartan Humphreys, John Ishiyama, Dean Karlan, Alan Kraut, Arthur Lupia, Patricia Mabry, Thomas Madon, Neil Malhotra, Evan Mayo-Wilson, Marcia McNutt, Edward Miguel, Elizabeth Levy Paluck, Uri Simonsohn, Courtney Soderberg, Barbara A. Spellman, Jay Turitto, Gary VandenBos, Simine Vazire, Eric J. Wagenmakers, Rick Wilson, and Tal Yarkoni. Promoting an open research culture. *Science*, 348(6242):1422–1425, 2015. doi: 10.1126/science.aab2374.

Daniel Nüst, Markus Konkol, Edzer Pebesma, Christian Kray, Marc Schutzeichel, Holger Przibytzin, and Jörg Lorenz. Opening the publication process with executable research compendia. *D-Lib Magazine*, 23(1/2), 2017. doi: 10.1045/january2017-nuest. URL https://www.dlib.org/dlib/january17/nuest/01nuest.html.

Open Source Security Foundation. Supply-chain levels for software artifacts version 1.2, 2026. URL https://slsa.dev/spec/v1.2/. Accessed 2026-06-30.

Matthew J. Page, Joanne E. McKenzie, Patrick M. Bossuyt, Isabelle Boutron, Tammy C. Hoffmann, Cynthia D. Mulrow, Larissa Shamseer, Jennifer M. Tetzlaff, Elie A. Akl, Sue E. Brennan, Roger Chou, Julie Glanville, Jeremy M. Grimshaw, Asbjorn Hrobjartsson, Manoj M. Lalu, Tianjing Li, Elizabeth W. Loder, Evan Mayo-Wilson, Steve McDonald, Luke A. McGuinness, Lesley A. Stewart, James Thomas, Andrea C. Tricco, Vivian A. Welch, Penny Whiting, and David Moher. The prisma 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372:n71, 2021. doi: 10.1136/bmj.n71.

Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.

Nathalie Percie du Sert, Viki Hurst, Amrita Ahluwalia, Sabina Alam, Marc T. Avey, Monya Baker, William J. Browne, Alejandra Clark, Innes C. Cuthill, Ulrich Dirnagl, Michael Emerson, Paul Garner, Stephen T. Holgate, David W. Howells, Natasha A. Karp, Stanley E. Lazic, Katie Lidster, Catriona J. MacCallum, Malcolm Macleod, Esther J. Pearl, Ole H. Petersen, Frances Rawle, Penny Reynolds, Kieron Rooney, Emily S. Sena, Shai D. Silberberg, Thomas Steckler, and Hanno Würbel. The arrive guidelines 2.0: Updated guidelines for reporting animal research. *PLOS Biology*, 18(7):e3000410, 2020. doi: 10.1371/journal.pbio.3000410.

Pliny the Elder. *The Natural History, Book XXXIII: The Natural History of Metals*. c. 77 CE. URL https://www.gutenberg.org/files/62704/62704-h/62704-h.htm. First-century source text; English translation by John Bostock and H. T. Riley consulted via Project Gutenberg.

Adam Rule, Amanda Birmingham, Cristal Zuniga, Ilkay Altintas, Shih-Cheng Huang, Rob Knight, Niema Moshiri, Mai H. Nguyen, Sara Brin Rosenthal, Fernando Perez, et al. Ten simple rules for writing and sharing computational analyses in jupyter notebooks. *PLOS Computational Biology*, 15(7):e1007007, 2019. doi: 10.1371/journal.pcbi.1007007.

Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

Kenneth F. Schulz, Douglas G. Altman, David Moher, and CONSORT Group. Consort 2010 statement: Updated guidelines for reporting parallel group randomised trials. *BMJ*, 340:c332, 2010. doi: 10.1136/bmj.c332.

Sigstore. Sigstore documentation, 2026. URL https://docs.sigstore.dev/. Accessed 2026-06-30.

Arfon M. Smith, Daniel S. Katz, Kyle E. Niemeyer, and FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*, 2:e86, 2016. doi: 10.7717/peerj-cs.86.

SPDX. Spdx specifications, 2026. URL https://spdx.dev/use/specifications/. Accessed 2026-06-30.

Victoria Stodden, Jonathan Borwein, and David H. Bailey. Setting the default to reproducible. *Computing in Science & Engineering*, 13(4):24–32, 2011. doi: 10.1109/MCSE.2011.36.

The Goldsmiths' Company Assay Office. History of hallmarking, 2026. URL https://www.assayofficelondon.co.uk/about-us/history-of-hallmarking. Official chronology consulted for pre-1800 hallmarking milestones, including 1238, 1300, 1478, 1576, 1677, and eighteenth-century standards.

UNESCO. Unesco recommendation on open science, 2021. URL https://www.unesco.org/en/open-science/about.

Erik von Elm, Douglas G. Altman, Matthias Egger, Stuart J. Pocock, Peter C. Gøtzsche, Jan P. Vandenbroucke, and STROBE Initiative. The strengthening the reporting of observational studies in epidemiology (strobe) statement: Guidelines for reporting observational studies. *The Lancet*, 370(9596):1453–1457, 2007. doi: 10.1016/S0140-6736(07)61602-X.

W. B. (William Badcock). *A Touch-Stone for Gold and Silver Wares, or, A Manual for Goldsmiths*. London, 1678. URL https://archive.org/details/0040TOUC. Internet Archive record for the goldsmiths' manual on fineness standards, statutes, weights, and counterfeit coin.

Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E. Bourne, et al. The fair guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18.